Bucharest University of Economic Studies
Faculty of Cybernetics, Statistics and Economic Informatics
Economic Informatics Program

Architecture and Evaluation of a SIEM Integrating Unsupervised Anomaly Detection and Knowledge-Grounded Language Models

BACHELOR THESIS

Coordinator
prof. univ. dr. CĂTĂLIN BOJA

Graduate
SECĂREA DIANA MARIA

Bucharest 2026

Declaration of Originality and Responsible Use of Artificial Intelligence

I, Secărea Diana Maria, hereby declare that this bachelor's thesis is my own original work and that the results, analyses, interpretations, and conclusions presented herein are entirely the result of my own intellectual effort, except where explicit reference is made to the work of other authors.

I confirm that all materials, ideas, data, figures, and information obtained from external sources, including books, journal articles, conference papers, websites, and other publications, have been properly acknowledged and cited in accordance with academic standards and are included in the bibliography.

Where Generative Artificial Intelligence (AI) tools were used during the preparation of this thesis, such use was limited to permitted support activities (such as language revision, translation, idea generation, or other disclosed purposes), was carried out in accordance with the university's guidelines, and has been transparently documented where required. I remain fully responsible for the content, accuracy, originality, methodology, analysis, and conclusions of this thesis.

I understand that any false declaration, plagiarism, misrepresentation of authorship, or undisclosed misuse of Generative AI tools may constitute a violation of academic integrity regulations and may result in disciplinary or legal consequences.

Contents

1. Introduction

The cybersecurity landscape is undergoing a profound, yet challenging, transformation. As businesses become more dependent on computerized infrastructure, the size and sophistication of cyber threats have grown to large proportions. Everyday companies are subjects of multiple cyberattacks from all over the world. The scale of modern attacks, from advanced persistent threats (APTs) to ransomware campaigns, has overcome traditional rule-based security monitoring, that became really insufficient in responding quickly and promptly enough.

Security Information and Event Management (SIEM) systems are essential to business monitoring because they centralize diverse logging events (e.g., security logs, network events, endpoint activity), normalize them, and support correlation and alerting processes. [1]. Among open-source SIEM solutions, Wazuh has become a leading platform, offering host-based intrusion detection, log analysis, file integrity monitoring, vulnerability detection, and regulatory compliance functionalities. [2] However, Wazuh and similar SIEM platforms share a fundamental limitation: they rely on rules that are predefined and signature-based detection, which leaves them oblivious when confronting zero-day or polymorphic threats.

This thesis addresses these limitations by proposing an AI-enhanced threat detection system that integrates machine learning approaches into the Wazuh SIEM platform. The principal design hypothesis points out that by integrating unsupervised anomaly scoring with grounded natural-language reasoning over a curated cyber knowledge (e.g., adversary TTP frameworks, standards-based threat intelligence, internal runbooks) we can reduce time spent to triage and improve analyst trust. [3] This is done only by taking into account the following: scalability and latency, distribution shift (concept drift), adversarial manipulation, and privacy/compliance obligations, things that are addressed below.

The first component utilizes an ensemble of Isolation Forest and Autoencoder, unsupervised anomaly detection algorithms that assign priority scores to security events based on their statistical deviation from normal system behavior. The second component that I implemented is a Retrieval-Augmented Generation (RAG) system that serves as an intelligent copilot. This system aims to provide SOC analysts with contextual threat intelligence by correlating Wazuh alerts to a knowledge base of known attack techniques, MITRE ATT&CK tactics, Sigma detection rules, YARA malware signatures, CISA CVEs, live IoCs.

The two Machine Learning models were trained using real alert data collected from a Wazuh deployment over multiple months. This was supplemented by training data obtained from a WordPress plugin on a website that was subjected to real-world attacks over a two-months period. This combination of operational data collected locally and external attack data ensures that the models capture both the normal behaviour of the monitored environment and the statistical patterns of real malicious activities.

The RAG system operates as a threat intelligence copilot by retrieving structured attack descriptions and using semantic similarity search to retrieve the most relevant explanations for each detected threat it finds. For high precision retrieval, the system uses sentence embeddings

(all-MiniLM-L6-v2), Qdrant vector database, BM25 keyword search (lexical), and rank fusion (for better retrieval confidence afterwards). After that, a large language model (LLM) then summarizes the retrieved context to give human-readable explanations and useful recommendations. It does this by following specific rules based on similarity, to select the best attack descriptions.

The subsequent sections are organized as follows. Chapter 2 provides domain analysis, including the advantages of Artificial Intelligence in cybersecurity, the SIEM ecosystem, the Wazuh platform, and the specific limitations this thesis wants to address. Chapter 3 describes the algorithms and methods used, including Isolation Forest, Autoencoder model, RAG, and hybrid retrieval. The architecture of the solution and its component design are also present there. Chapter 4 details the implementation, including training pipelines, RAG ingestion, and Wazuh integration, while Chapter 5 offers conclusions and directions for future research.

2. AI in Cybersecurity: The Problem Domain

Businesses, governments, and essential infrastructure have all gone digital; therefore, they created an area that is vulnerable to attacks of large scale and complexity. The 2024 Global Risk Report published by the World Economic Forum identifies cybersecurity failure as one of the top ten global hazards in terms of probability in a firm [4]. Despite the limited effort of analysts to protect the infrastructure, modern hackers use smart tactics that bypass traditional detection methods, including supply-chain compromises, living-off-the-land binaries (LOLBins), fileless malware, and AI-generated phishing. This dynamic nature of cyber threats requires constant adaptation in malware detection systems, increasing the focused effort of humanity to stay at the forefront of technological advancement. [5] Therefore, as AI and ML become tools of the automated attacks, it is crucial that they will be integrated in the defensive solution as well, allowing systems to adapt to threats evolving in real-time. In fact, it is recognized that by making use of AI-driven cybersecurity solutions, the scanning of enormous amounts of real-time data will be ensured. The systems will uncover unusual patterns that can indicate security breaches and enhance detection correctness. They will then adopt proactive defense mechanisms that stop new threats before they escalate into full-blown attacks. [6]

One of the most major obstacles in the cybersecurity landscape consists of “alert fatigue” in Security Operations Centers, where analysts are struggling every day with wide volumes of alerts produced by SIEM solutions that they are not capable of handling. Industry data show that SOCs are inundated with nearby 4,490 warnings every day, on average, while around 67% cannot even be addressed because of the lack of analyst bandwidth. Here artificial intelligence comes in hand, enabling analysts to focus on the most critical threats and respond more efficiently to potential security breaches. [7] Therefore, to strengthen SOC capabilities, automating alert triage is essential. This involves prioritizing high-risk incidents and reducing the frequency of false positives.

In essence, the advancement of AI represents a new step into the world of cybersecurity, bringing a new paradigm of thinking and new solutions to explore.

One advantage is automated prioritization. AI systems can score and rank thousands of security events in real time, so that analyst may focus on the most important warnings first. This fights alert fatigue directly and reduces the mean time to detect and the mean time to reply. [8] Machine learning evaluation techniques are designed to evaluate the rate of true and false positives on the attack surface, therefore tackling only what is right.

Another relevant advantage is Contextual Intelligence. RAG systems can turn raw alarm data into actionable intelligence by automatically pulling pertinent threat context from knowledge bases. This makes it easier for SOC analysts to make decisions quickly and with more information.

An alternative consideration is scalability. The amount of computer resources needed for AI-driven analysis grows linearly, while the number of people needed for human analysis grows also linearly. Companies can handle millions of events per day without having to hire more people.

Also, we have to consider continuous learning. It is possible to retrain machine learning models on new data every so often so that they can react to changing threats and changes in your organization. This capacity to adapt is a big plus over static rule sets that need to be updated by hand. By learning patterns in time, network activity and even authentication sequences, they can have access across all the dimensions and “see” more than humans, in improved ways and shorter times.

Those advantages are now part of our present and we have to use them as such. Comparing them to what conventional cybersecurity solutions, the new algorithms used by AI/ML are now able to make companies grow and prevent new attacks that were not yet recognized.

2.1. Security Information and Event Management (SIEM)

Security Information and Event Management system is an on-premise platform that gathers security events from multiple sources within an organization’s IT infrastructure, correlates and integrates them. The main goal of SIEM systems is to turn raw, heterogeneous events into useful information, often called actionable intelligence. [1] SIEM platforms are the operational backbone of Security Operations Centers, providing capabilities like log management, real-time monitoring, alerting, forensic analysis, and regulatory compliance reporting.

A SIEM system's core functions can be broken down into the following components:

Real-time Log Collection and Aggregation is a capability that is generally well known. SIEM agents are installed on endpoints, servers, network devices, and cloud services, and they gather security-relevant logs and transmit them to a central manager for refinement. Such architectures are argued to improve scalability and timely availability of data for downstream analytics such as correlation and anomaly detection in recent works. Centralized log management keeps system activity logs consistent and synchronized, improving data integrity and helping forensic investigations. Recent work by Marios Vardalachakis shows that log aggregation is not only vital for real-time threat detection but also provides the data foundation for more advanced SIEM capabilities, including behavioral analysis, compliance reporting, and incident response orchestration. [9]

Another interesting component is Event Normalization: Raw log data from heterogeneous sources (for example syslog, Windows Event Logs, application logs, network flow data) is normalized into a common schema to allow correlation across sources.

Correlation & Rule Engine Patterns are identified using correlation logic and pre-defined rules. For instance, a brute-force detection rule could be triggered by multiple unsuccessful SSH login attempts from a single IP address within a time frame.

Alerting and Dashboards is also a concern for recent open source SIEM solutions: Correlation rules are satisfied, and SIEM raises alerts with severity levels, which are then presented through dashboards for analyst review.

Commercial SIEM solutions like Splunk, IBM QRadar and Microsoft Sentinel dominate the enterprise market, but at a considerable licensing cost. Open-source alternatives,

in particular Wazuh, have grown substantially in popularity as they provide similar capabilities without the licensing costs and bring enterprise-grade security monitoring to organizations of all sizes.

Wazuh is a free, open-source security platform that provides integrated Extended Detection and Response (XDR) and SIEM capabilities. Initially developed from OSSEC in 2015, Wazuh has since evolved into one of the most extensively used open-source and security solutions, with over 20 million downloads and an active community of contributors. It is used by organizations of all kinds. They range from small firms to Fortune 500 enterprises, using it as a cost-effective alternative to commercial SIEM platforms such as Splunk, IBM QRadar, and Microsoft Sentinel. [10]

In the Appendix, the first figure represents an actual alert captured from the project deployment on February 16, 2026, representing a kernel-level rootkit detection. This alert shows some important aspects of Wazuh's output format:

- Severity Level 11 (out of 15): This is a high-severity alert. For example, Wazuh assigns level 11 to events that indicate a "possible kernel level rootkit," showing how serious rootkit-class threats are compared to other lower-level attacks.
- MITRE ATT&CK Mapping: The rule automatically maps to T1014 (Rootkit) under the Defense Evasion tactic. This demonstrates Wazuh's native ATT&CK integration, which is what the RAG system uses to pull threat intelligence from.
- Rootcheck Detection Logic: The alert was triggered because the file /tmp/.mount_CursorVGCXRH was "hidden from stats but showing up on readdir," which is a classic kernel rootkit behaviour. It shows a malicious kernel module intercepts the stat() system call to hide files. In the meantime the readdir() call remains not affected.
- Compliance Annotations: Though we do not show that in the shortened example, most Wazuh rules include PCI-DSS, GDPR, HIPAA, and NIST 800-53 references, so they can be reported for automatic compliance reporting.

Limitations of Traditional SIEM Platforms and Wazuh

Despite its entire feature set and active development, Wazuh, along with other traditional SIEM platforms, show us several underlying limitations that are characteristic to the rule-based detection paradigm. Considerable research has been devoted to understanding those limitations and proposing solutions, ideas that directly motivate the AI enhancements proposed in this thesis.

The most critical disadvantage of any rule-based SIEM is its fundamental inability to detect threats for which no rule exists and unknown attack patterns. Wazuh can only detect events that match predefined patterns encoded in its XML rule definitions. A new attack technique, a zero-day exploit, or a sophisticated adversary using custom tools will create events that do not match any existing rule ID, so these attacks can go undetected. [11] This is not a deficiency specific to Wazuh, but a structural limitation of the signature-based SIEMs: the system is reactive by design, requiring a human analyst to write a new rule after each new attack is discovered. In a landscape where new CVEs are found daily, this approach is always

one step behind the attacker. So far, investigations have been restrained to show that rule-based correlation is no longer sufficient in protecting an IT environment from continuously evolving dynamic security threats. There is an increasing demand to complement rule-based correlation with more advanced techniques, such as anomaly detection. In contrast to rule-based systems, anomaly detection does not rely on statically created rules. Aggregated events are analysed with a multivariate unsupervised anomaly detection algorithm and an anomaly score is assigned to each instance, therefore allowing the discovery of every new pattern. [12]

The high false-positive rate from static thresholds is a well-documented limitation. Wazuh's correlation rules use statically defined ceilings (e.g., "more than 8 failed logins in 120 seconds"). These thresholds are designed to be commonly applicable but cannot account for the specific characteristics of each deployment environment. An organization with 10,000 employees will naturally see more failed login attempts than a team of 10, yet the same threshold applies to both. Additionally, legitimate automated processes (service accounts, monitoring tools, CI/CD pipelines) can trigger authentication-based rules during normal operation, generating false positives that confuse analysts and waste investigative resources. Previous researches done by Desiree Sacher [13] has tended to focus on mitigations that involve applying sets of resolution categories for security incidents or recommended follow-up action, such as updating suppression rules, correcting SIEM or device configurations, yet none point out to the beneficial use of both AI and ML.

Another limitation that has been claimed is the lack of contextual threat intelligence. SIEMs sends alerts with the raw event data (log message, decoded fields, rule description, severity level, and compliance annotations), but without any contextual explanation of the threat. If an analyst receives an alert for “Possible kernel level rootkit” (rule 521), they have to research what rootkits are, how T1014 works, what indicators to look for, and what remediation steps to take on their own. Manual research can be time-consuming and prone to high errors, especially for junior analysts or organizations that don’t have dedicated threat intelligence teams. Therefore, a major bottleneck in incident response is the gap between alert creation and actionable understanding.

2.2. Anomaly detection and AI in SIEMs

Anomaly detection refers to the identification of complex trends and patterns in data that do not conform to expected behavior. In the literature, such patterns are perceived as anomalies or outliers, terms which are used interchangeably in this context of attack spotting. The basic idea of anomaly detection is to define a region that represents normal, expected behavior and classify anything else as anomalous. [14]

Anomaly detection has been applied in many application domains such as intrusion detection, fraud detection and image processing and has been researched extensively. Studies done by different AI pioneers like G. Pang et al. [15] describe how anomaly detection has improved significantly over the past years, revolutionizing it with deep learning and enabling it to be used for more complex detection tasks in complex data environments.

The best SIEM programs don’t use only anomaly detection to find threats, instead they also use predictable detection. They use both together to ensure a more reliable threat

identification. Deterministic rules are performing properly at finding notorious attack patterns and hostile behaviors, while anomaly detection can also find novel strange behaviors that could be signs of never-seen-before threats. Recent developments have signified a substantial transition from conventional rule-based methodologies to the implementation of machine learning approaches, especially the utilization of language models.

Jonathan Pan et al [16] present a vector-based method that uses the Retrieval Augmented Generative (RAG) model to analyse log entries by searching an online database of normal logs. They concentrate on creating a vector of fundamental logs that are subsequently analysed by many LLMs. Nonetheless, the research is lacking a methodology to prioritize and interpret the genuine true positive attacks, resulting in excessive resource consumption and execution latency while giving the LLM a single log entry at a time.

On the other hand, the research made by Rohit Arora et al [17] propose the use of Isolation Forest in detecting anomalies only in the fields of firewall log data. The model is trained on 12 characteristics pertaining to specific firewall aspects (IPs, bytes, actions, elapsed time, packets transmitted, and packets received) and evaluates whether Isolation Forest is the most accurate method for huge datasets. Nevertheless, upon the discovery of new anomalies, these unsupervised models require retraining to assimilate the new information and will demand substantial resources to maintain resilience against future alterations. The research argues that the models are not resilient to future change and cannot be fully autonomous, as they constantly involve retraining. It is also lacking a second ML algorithm to balance the scores.

Recent projects done in this area demonstrate a log anomaly detection platform (shown publicly in 2025) that pairs Isolation Forest–based anomaly detection and severity presentation with a minimal LLM explanation layer. Additionally, it features a dedicated vector database component that archives embeddings of "normal logs" and anomalies, facilitating retrieval for grounding and comparison. [18] It is important to note that an attack pattern or rule can be mapped to multiple techniques, and a single technique may fall under multiple tactics, further complicating the mapping process. This limitation may restrict the method’s applicability in real-world scenarios where comprehensive coverage of TTP classes is essential and a rule-based mapping might be insufficient in getting a clear vision over the real threat. Also, any hacker that knows that the host is using a single machine learning model can easily study the model's decision boundary and create adversarial inputs that fall below the detection threshold. Therefore, studying the anomalous behaviour can be complicated with only one algorithm.

Another one focuses on network-traffic “SOC agent”, using Isolation Forest to compute an anomaly score and then produces a natural-language “reasoning” string with MITRE ATT&CK technique mapping. [19] It continuously ingests packet or flow-level data, applies anomaly detection techniques to flag unusual patterns, and then leverages an LLM to interpret those anomalies. However, this is not a dynamic or retrieval-based process. Instead, it relies on predefined mapping logic that links detected patterns to specific ATT&CK techniques. The system is deliberately simplistic and lacks advanced threat intelligence or AI pipeline functionalities: no vector database, no retrieval step, no semantic search, no application of

Sigma/YARA/STIX rules, and no Retrieval-Augmented Generation (RAG) pipeline. Consequently, the LLM may readily hallucinate due to the absence of information foundation.

While these systems detect anomalous behaviour in traffic and chain them to an attack technique (if pattern == "port scan": technique = "T1046"), my thesis is planning to create a real threat intelligence that focus on network features to ensure a full anomaly detection used to prioritize, score, explain and execute. It is intended to be more than a static mapping table, but a real full vector-search retrieval step that explain the anomalies found in the whole system. Here Machine Learning and Artificial Intelligence are chained together to their maximum capacities to function similarly to a human threat researcher.

3. Methods and Algorithms Used

This chapter presents the technical essence of the system: the methods and algorithms that transform raw Wazuh security alerts into actionable threat intelligence. The approach combines two complementary paradigms. The first is an unsupervised machine learning pipeline for anomaly detection, designed to flag suspicious activity without relying on labelled attack signatures, a deliberate choice given that real-world security environments are dominated by previously unseen threats. The second is a Retrieval-Augmented Generation (RAG) system that grounds a large language model in an external, continuously updated knowledge base (that updates automatically), that enable contextual reasoning over the detected anomalies.

3.1. System Diagram

The general architecture diagram provides a big overview of the entire system, showing all major components and their relationships. It illustrates the entire architecture, where is presented the Endpoint Layer, Manager Layer, AI Threat Hunting Copilot (the new integrated part), the Storage Layer, and Visualization Layer.

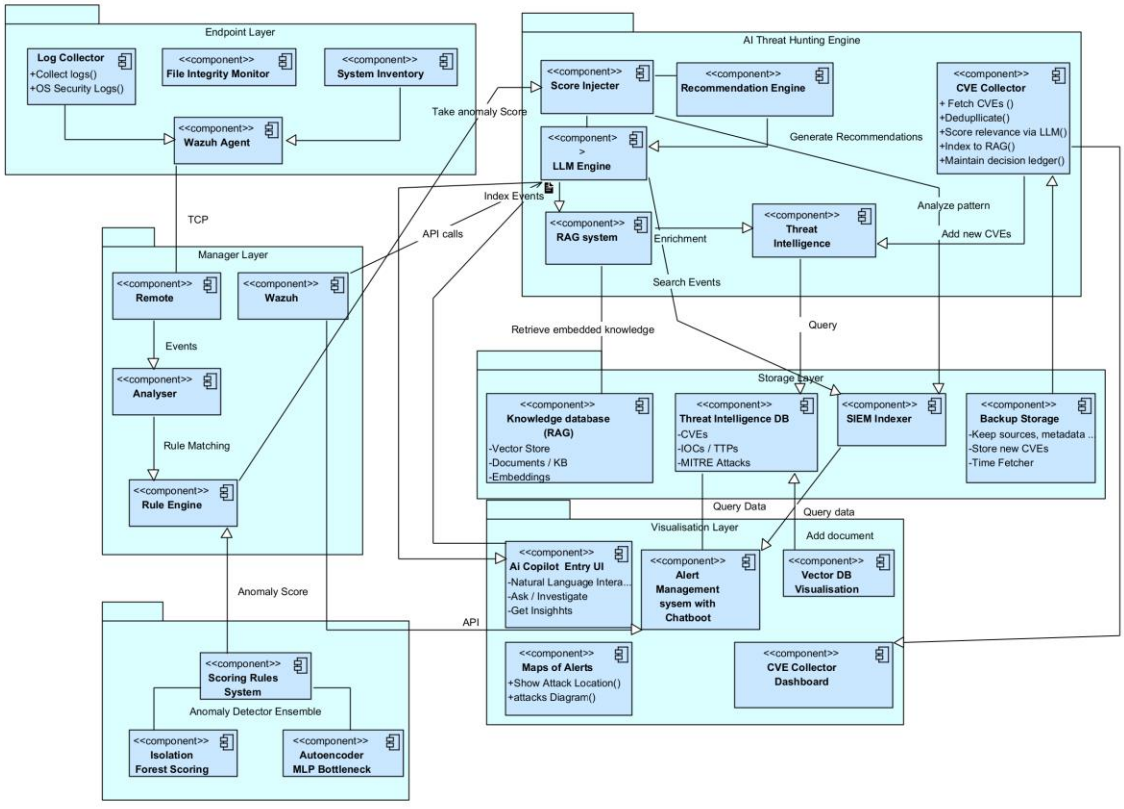


Figure 3.1.1. Architecture of the Proposed System (with SIEM Integration).

This sequence diagram details the step-by-step interaction between components during log collection and processing. It shows the message flow from System through Agent, Log Collector, Remote Analysis, Rule Engine, Wazuh DB, AI Threat Engine, and Storage.

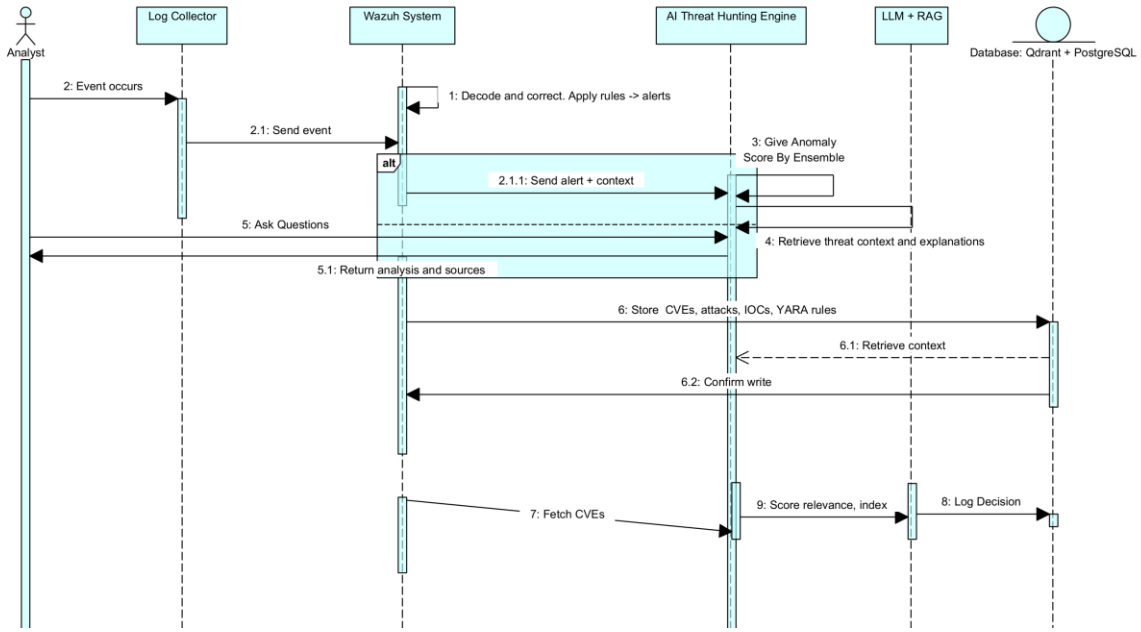


Figure 3.1.2. Interaction diagram.

The interaction diagram models the system's runtime behaviour. Wazuh decodes incoming events and applies rule-based correlation to generate an alert, which is forwarded with its context to the AI Threat Engine, the Engine takes the ML + RAG components to take out threat intelligence and compute an anomaly score, then updates the enriched alert to Storage. The analyst submits a question to the Engine, generates a grounded response via RAG, and returns the answer together with its supporting sources.

3.2. Machine Learning Approach

The ML pipeline is presented as a progression from individual models to a valid system. We begin with the Isolation Forest as a lightweight baseline detector, then extend it to the cases of unknown attack types to test its limits. To compensate for those limits, we introduce a complementary bottleneck MLP Autoencoder, which captures different anomalous patterns. Since the two detectors are complementary, we fuse them into an ensemble.

Isolation Forest for Anomaly Detection

Isolation Forest is an unsupervised machine learning algorithm specifically designed for anomaly detection, introduced by Liu, Ting, and Zhou in 2008. [20] Unlike traditional anomaly detection methods that build a profile of normal behavior and then identify deviations, Isolation Forest isolates anomalies directly by taking advantage of the basic property that anomalous data are “few and different,” and thus easier to be separated from the rest of the data.

The algorithm generates a collection of Isolation Trees (iTrees) by recursively dividing a dataset into smaller subsets through the use of randomly chosen characteristics. Then, it is used to split values, thereby isolating anomalies nearer to the root due to their uniqueness. Anomalies are recognized as instances with shorter average path lengths within the iTrees, as their distinctive attributes make them more susceptible to isolation.

The anomaly score for a data point x is computed as:

$$s(x,n) = 2^{\left\{-\backslash frac\{E(h(x))\}\{c(n)\}\right\}}$$

Equation (3.1)

E(h(x)) represents the average path length of x over all trees in the ensemble, and c(n) which serves as a normalization factor, denotes the average path length of unsuccessful searches in a Binary Search Tree containing n nodes. Abnormalities are shown by scores close to 1, normal values are indicated by scores close to 0.5, and very normal points are indicated by scores below 0.5. [20]

For this project, the hyperparameters for the Isolation Forest model are as follows:

Table 3.2.1. Hyperparameters of Isolation Forest

Parameter	Value	Rationale
n_estimators	100	Number of isolation trees; 100 strikes to be a good balance between accuracy and speed
contamination	0.1	Expected proportion of anomalies (10%); tuned using observed distribution of alerts
random_state	42	Ensures reproducible results across training runs
n_jobs	-1	Uses all available CPU cores for parallel tree construction. There were tests using Nvidia GPU 5070 as well.

The cybersecurity threat landscape is defined by its continuous and harsh evolution. New vulnerabilities are discovered daily: the NIST National Vulnerability Database registered over 28,000 new CVEs in 2023 alone, a 15% increase over the previous year [21]. This risk increases for open-source contributions, that lack formal security training and heavily rely on community-driven code reviews. [22] Since attackers continuously develop novel exploitation methods, whenever there is a new exploit, it will not resemble the previously catalogued threats. In this environment, any detection system that relies exclusively on signatures or predefined rules is overly faulted: it can only detect what it has already seen. This is the fundamental limitation that makes Isolation Forest an exceptionally powerful choice for security anomaly detection.

Because Isolation Forest is an unsupervised algorithm that learns the statistical distribution of normal behavior, rather than remembering attack signatures, it is therefore capable of detecting previously unknown/ undefined attacks. For example, if an unknown threat produces an alert with characteristics different from the learned baseline, for instance an unusual combination of off-hours activity, failed authentication counts, and multiple IP addresses, the model assigns a high anomaly score. This happens regardless of whether that that particular attack pattern has ever been seen before. This property allows the Isolation Forest implementation in this project, having an ensemble with an Autoencoder, to be more

than just a simple classifier of known attacks, but also an early warning system for zero-day threats that would completely bypass rule-based SIEM detection.

This forward-looking capability is particularly relevant in the context of the MITRE ATT&CK framework, which currently documents over 200 techniques and 400 sub-techniques. New techniques are added regularly as adversary tradecraft evolves. While the RAG component of this system maps alerts to known ATT&CK techniques, the Isolation Forest combined with Autoencoder component serves as a safety net that catches anomalous behavior even when no matching ATT&CK technique exists in the knowledge base. The score generated is injected into the formatted alert context, so any benign or real alerts can be properly defined before being explained. Together, the 2 components provide defense-in-depth: known threats are explained by RAG, and unknown threats are flagged by the anomaly detector.

Autoencoder: Bottleneck MLP

An autoencoder [23] is a neural network trained to reconstruct its own input. It consists of two sides: an encoder that compresses the input into a lower-dimensional latent representation, and a decoder that reconstructs the original input from that compressed code. Basically, it extracts the relevant information after the encoder, so that it can use it to solve the problem in the decoder. The network is trained to minimise reconstruction error on the training set. Formally, for input $x \in \mathbb{R}^d$:

$z = f_{\{enc\}}(x) \in \mathbb{R}^k$, $k < d$, where $f_{\{enc\}}$ is the encoder and k*k* is the bottleneck dimension.

$\hat{x} = f_{\{dec\}}(z) \in \mathbb{R}^d$, where $f\_\{dec\}$ is the decoder.

$L(x) = ||x - \hat{x}||^{\{2\}}$, where $L(x)$ is the mean squared reconstruction error (MSE).

Equation (3.2)

In this implementation, a Multilayer Perceptron (MLP) autoencoder with a symmetric bottleneck architecture is used: $16 \rightarrow 8 \rightarrow 4 \rightarrow 8 \rightarrow 16$. The input and output dimension (16) matches the shared feature vector described below. The bottleneck dimension (4) is 25% of the input, chosen to force the encoder to eliminate noise while retaining. ReLu function is used to determine what output a neuron will output, in order to learn patterns lately. [24] ReLU activation is applied at hidden layers; the output layer uses a linear (identity) activation to allow unconstrained reconstruction in the feature space.

The implementation uses scikit-learn's MLPRegressor configured in the autoencoder mode (to reconstruct itself) so the target is identical to the input: $Input = X_{\{scaled\}}$, $Target = X_{\{scaled\}}$. Therefore, it applies a scalar to an x matrix, to modify the data. This allows the full scikit-learn pipeline (StandardScaler → MLPRegressor) to be serialised with joblib and loaded identically to the Isolation Forest artefact, simplifying deployment and model management.

Ensemble

The Isolation Forest and the Autoencoder are algorithmically complementary and they succeed and fail in opposite ways. The Isolation Forest detects anomalies through geometric

isolation: a point is anomalous if it can be separated from the rest of the data quickly by random axis-aligned cuts. This makes it highly sensitive to outliers, as an alert with an extreme rule_level or an unusual failed_count will be isolated in few steps regardless of other features. This is where the other component comes in.

The Autoencoder detects anomalies through failure in reconstructing them: a point is anomalous if the network cannot reproduce it accurately, after it learned its representation of normality. If the data to reconstruct it is too little, then the Autoencoder cannot learn it properly. This makes it sensitive to unusual combinations of features, even when no individual feature is extreme. An alert where rule_level=7 (present in clean data), hour=3 (present in clean data), and privileged_account_change=1 (rare but present) might not be isolated early by the Isolation Forest, yet the specific triple combination may never have appeared in the clean training set, causing the autoencoder to reconstruct it poorly. If it cannot reconstruct it, then it's an attack.

This complementarity means that the two models do not simply agree on a specific threshold, they genuinely disagree on some alerts. An alert that the Isolation Forest flags but the Autoencoder does not (or vice versa), carries different information than an alert both flag. The ensemble tier system encodes this information explicitly, making use of both.

Moreover, an attacker who knows the engine is running IF can craft slow, low-volume attacks that blend with the rest of the alerts such that they never get isolated quickly. An attacker who knows the engine is running AE can craft attacks whose features mimics the diversity such that the reconstruction error stays low. But both evasion strategies are contradictory: blending into density means being statistically average (high reconstruction fidelity); mimicking the normal manifold means having common feature combinations (easy to isolate). You cannot simultaneously evade them by having a simple attack strategy, as they complement each other. Therefore, a more valuable SIEM platform will benefit from both, combined.

Comparison with Alternative Anomaly Detection Methods

Research names several machine learning algorithms that could have been applied to cybersecurity anomaly detection. The choice of Isolation Forest and Autoencoder for this project was made after careful evaluation against the most promising alternatives. The following comparison explains why the ensemble is the best candidate for a task like SIEM alert prioritization:

Table 3.2.2. Comparison of Anomaly Detection Methods for SIEM Alert Scoring

Method	Strengths	Weaknesses for SIEM Context	Verdict
Isolation Forest	Linear time complexity O(n); no density estimation needed; handles high-dimensional data; robust to irrelevant features; does not assume data distribution	Requires sufficient training data volume; random splitting can miss subtle patterns in very low-dimensional spaces	SELECTED: Best fit in ensemble
One-Class SVM	Strong theoretical foundations; effective for	$O(n^2)$ to $O(n^3)$ training complexity makes it impractical for large SIEM	Rejected: Scalability

	well-separated normal/anomaly boundaries	datasets; sensitive to kernel choice; poor scalability	
Local Outlier Factor (LOF)	Captures local density variations; detects anomalies in clusters of varying density	Requires computing k-nearest neighbors for every point $O(n^2)$; does not produce a model that can score new points without retraining; memory-intensive	Rejected: No model persistence
Autoencoders (Deep Learning)	Can learn complex nonlinear patterns; flexible architecture	Requires large labeled or semi-labeled datasets to perform better, yet it can learn from unlabeled data as well; training is computationally expensive (GPU recommended); hyperparameter tuning is complex; interpretability is poor especially when data is messy (complex)	SELECTED: Best fit in ensemble
Gaussian Mixture Models	Probabilistic framework; provides density estimation	Assumes data follows Gaussian distributions; poor performance on non-Gaussian security data; sensitive to number of components	Rejected: Distribution assumption
DBSCAN	Discovers clusters of arbitrary shape; identifies noise points as anomalies	Sensitive to epsilon and min_samples parameters; struggles with varying density; $O(n^2)$ without spatial indexing; does not produce anomaly scores	Rejected: No scoring output

In this thesis, the model is trained on complete, unlabeled data without any prefiltering of attacks events. The Isolation Forest is trained with contamination='auto', making no assumption about the proportion of anomalies in the data (it uses random recursive splitting). The Autoencoder is trained via iterative self-cleaning: in the first round the model trains on all alerts and scores each by reconstruction error, then discard based on errors. This strategy aligns with unsupervised anomaly detection for the Isolation Forest and robust unsupervised with iterative trimming for the Autoencoder.

Feature Engineering

The model takes 16 numerical features from each Wazuh alert for scoring:

• Event Frequency / Word Count (f1): Word count of alert message (limited to 60) is a proxy of event complexity. Attack-related events, such as reverse shells or concatenated commands often generate longer messages. The cap prevents overly long and otherwise inoffensive reports from having a disproportionate impact on the score. Limitation: Short alerts can still be malicious, when coupled with other indicators.

• Log Size (f2): Total alert size in bytes or characters. A JSON object. Larger alerts, for example, often contain richer contextual information such as source IPs, ports, usernames or commands, which are characteristic of well-executed attacks. Limitation: Some valid modules (for example, audit or FIM logs) inherently generate large events, but they are clearly not attacks.

• Failed Attempts Counter (f3): Counts instances of keywords like failed, denied, error and invalid within the alert text. This is effective for detecting brute-force attempts, authentication failures or permission violations. Limitation: These terms can be found in benign audit messages, not necessarily attacks.

• Hour of Day (f4): Extracted from the event timestamp (0-23). Malicious automated activity usually happens at odd hours, not normal organizational behavior. Limitation: Effectiveness is reduced by time zone differences or 24/7 operational teams.

• Off-Hours Flag (f5): Binary feature indicating if the event occurred outside normal business hours (e.g., between 02:00–06:00 or outside 08:00–18:00). It makes the detection of temporal anomaly easier for the model. Limitation: this is not fundamentally adapted for organizations working continuously.

• IP Address Count (f6): Counts distinct IP addresses from fields like agent.ip, data.srcip. A large number of IPs could indicate scanning, distributed attacks or rotating proxies. Limitation: Proper CDN or Proxy infrastructure can also generate multiple IPs. Usually, these networks generate hundreds or even thousands of Ips.

• Port Count (f7): It counts the number of ports that are referenced in the structured fields or derived using text heuristics. Multiple ports may indicate reconnaissance or port scanning activity. Limitation: Firewall inventory or network monitoring alerts may, by nature, refer to many ports.

• Process / Execution Count (f8): Counts the number of times the keywords process, exec, spawn, bash, python or sh -c are present. Reverse shells, command injection and remote execution attacks generally trigger this feature. Limitation: Similar patterns may be generated by CI/CD systems or automation servers in normal operations.

• Wazuh Rule Severity Level (f9): Severity of Wazuh rules (0-15). Higher levels are alerts that the SIEM engine already considers as critical. This feature is a powerful contextual signal and not a direct decision signal. Limitation: severity is not always related to real world risk, as it does not ultimately show a real attack.

• Wazuh Rule ID (f10): Numeric identifier of the Wazuh rule that was triggered. Some rule IDs are highly correlated with malicious activity, enabling the model to learn attack-specific patterns. Limitation: distributions might be affected by new or custom rules without retraining. They are not supposed to have rule IDs.

• Suspicious Group Match Count (f11): This is the count of alert groups that overlap with a configurable list of suspicious categories given by analysts. This allows to tune the operations without re-training the model. Limitation: Needs active maintenance to prevent over-classification.

• Structured Data Field Count (f12): The number of fields in the data section of the alert that have been populated. Attack-related alerts often have more structured metadata. Limit different Wazuh modules populate fields differently.

• Public Source IP Indicator (f13): Binary feature indicating if source IP is publicly routable and not within trusted CDN ranges like Cloudflare. Communications with external public IP addresses may suggest exposure to the Internet or interaction with an attacker. Limitation: CDN lists need to be constantly updated.

• Suspicious URL Pattern Matching (f14): Searches for web attack indicators such as ../, ?cmd=, eval(, union select, <script>, wget, or curl -s using regular expressions. It's useful to detect injection attempts and exploit patterns. Limitation: Smarter attackers can make obfuscated payloads to evade simple regex detection.

• Unknown User Indicator (f15): Indicates authentication attempts that try to log in as an invalid or non-existent account (e.g., invalid user, unknown user, no such user). It's a good indication of brute-force or dictionary attack attempts. Limitation: Authentication systems use a variety of message formats, so some variants may be missed.

• Privileged Change Indicator (f16): Detects privileged or high-risk system changes such as group changes, promiscuous mode activation, root account manipulation or user/group management operations. According to specific Wazuh Rule IDs and alert collections. Limitation: need to be updated as new Wazuh rules are added.

In addition to the base anomaly score, the AI Threat Engine implements a multi-dimensional scoring system that combines the Isolation Forest and Autoencoder output with contextual factors. The combined priority score is calculated as follows:

$$Combined = 0.5 \cdot Anomaly\ Score + 0.2 \cdot Time\ Score + 0.2 \cdot Frequency\ score + 0.1 \cdot Network\ Score$$

Equation (3.3)

In essence, the 16 features compute only the anomaly score. The rest of the features (time score, frequency, network) are computed in this formula to give more weight to them, beyond what the model already learned. They are intended to be adapted to real-life production systems, that have their own particularities in scoring.

Therefore, the time score assigns higher weights to events occurring during off-hours (02:00-06:00: 30 points, 22:00-02:00: 20 points, business hours: 5 points), so that it shows that legitimate system activity is mostly concentrated during working hours while malicious activity often occurs outside these windows. Since this project is intended to be used in production, time-sensitivity is deemed necessary.

Ablation Study

I conducted an ablation study by progressively removing or adding one feature at a time and re-evaluating the model on the full unlabeled dataset. The baseline is the 16-feature model described above. Results are expressed as the change in F1-score and false positive rate relative to the baseline.

Because retraining is fast (< 5 seconds on the approximately 2000-sample set), each ablation variant was fully retrained from scratch rather than using approximate perturbation methods. This provides the best combination of features to train the model, taking into account not only Wazuh alert features but also peculiarities of new possible attacks.

The entire feature ablation can be found in Appendix as Table 5. The key findings will be described in the next part. Consequently, the three main features responsible for the majority of the model's discriminative power are:

1. privileged_account_change is the highest-impact feature (−6.3% F1 when removed). For example, the creation of new users/groups or mode events that are sparse in normal operations might be a high indicator of attacker persistence activity. It is observed that without this feature, these attack types drop to 0% detection.
2. failed_count is the primary signal for credential-attack bursts (−5.7% F1). Its CIS-aware zeroing means clean scan alerts do not count towards this total of attacks, which would lead otherwise to false positives.
3. rule_level provides a global severity prior (−4.1% F1). High-severity Wazuh rules are genuinely less common in normal operations, and their presence strongly co-occurs with attack events.

It is easy to point out that mitre_count experiment is particularly informative but not useful to consider: adding a feature that appears semantically relevant (MITRE ATT&CK technique count) actually decrease performance severely (+8.9% FP rate). This is because the feature distribution differs between the training environment (Wazuh without MITRE mapping) and production (Wazuh with MITRE rules enabled), something that might silently shift the FP rate.

I. Isolation Forest Scoring Mechanism

Let x be the feature vector of an alert. After z-score scaling on each dimension, the model returns $s = decision\_function(x)$. It returns a number like -0.12 or +0.05. Smaller values of s indicate more pronounced anomaly. To interpret it better, bounds [s_min, s_max] are calculated based on percentiles of raw scores from training data, then inversely mapped to normalized score N ∈ [0, 100]. So that large N refers to more anomalous behavior. Therefore, in this implemented system, the raw Isolation Forest output is transformed to improve interpretability.

II. Reconstruction Error as Anomaly Score

Instead of isolation, it reconstructs the alert. The raw anomaly score for alert x is the per-sample mean squared reconstruction error across all 16 features after z-score scaling:

$$recon\_error(x) = 1\backslash 16 \sum_{i=1}^{16} \left(x_{\{i,scaled\}} - \widehat{x_{\{i,scaled\}}}\right)$$

Equation (3.4)

This raw error is in the scaled feature space, not the original feature space. Operating in the scaled space makes sure that all 16 features contribute equally to the reconstruction loss regardless of their original importance. The reconstruction error is not directly interpretable as a 0–100 score, since its absolute magnitude depends on the training distribution. A calibration step, maps it to the same [0, 100] scale used by the Isolation Forest score, enabling direct comparison and ensemble combination.

III. Final Normalisation

To produce a human-readable score in [0, 100], where 100 is maximally anomalous, the raw reconstruction error and the isolation score are both calibrated using the 2nd and 98th percentiles of the observed reconstruction errors from the training set:

$$score_{norm(x)} = clip\left(\frac{recon_{error(x)} - err_{min}}{err_{\max} - err_{min}} \times 100, 0, 100\right)$$

Equation (3.5)

where err_min = percentile$_2$(errors on clean data) and err_max = percentile$_{98}$(errors on clean data). Using percentiles rather than the absolute minimum and maximum provides resistance against outliers in this training set (a single difficult-to-reconstruct clean alert should not compress the entire score range). Scores above 100 are cut, meaning alerts whose reconstruction error exceeds the 98th-percentile boundary of clean errors receive the maximum score of 100.

This calibration range is computed once at training time, stored inside the model artefacts (isolation_forest.pkl, autoencoder_model.pkl), and it is taken and applied identically at inference time.

Evaluation Metrics

Standard binary classification metrics are computed against the unlabeled dataset. An alert is classified as an anomaly if score_norm(x) ≥ θ.

Table 3.2.3. Evaluation metrics and their operational interpretation.

Metric	Formula	Interpretation
Precision	TP / (TP + FP)	Of all flagged alerts, what fraction are genuine attacks
Recall (TPR)	TP / (TP + FN)	Of all attacks, what fraction were caught
F1-Score	2 × P × R / (P + R)	Harmonic mean of precision and recall
False Positive Rate	FP / (FP + TN)	Fraction of benign alerts incorrectly flagged
Score Separation	μ_attack − μ_clean	Mean score gap between attack and clean populations

3.3. Retrieval-Augmented Generation (RAG)

It has been extensively studied in recent years that inaccurate annotation of SIEM rules can result in the misinterpretation of attacks, increasing the likelihood that threats will be ignored or not considered. RAG [25] was introduced by Patrick Lewis and a team of researchers

in a well-known seminar paper. RAG is an AI framework that improves Large Language Models (LLMs) by giving them access to external knowledge bases. By incorporating MITRE ATT&CK techniques, it enables analysts to categorize potential attack flows. Such mapping is proven to increase security professionals' ability to anticipate and mitigate the strategies employed by cyber adversaries. Solely relying on the implicit knowledge of LLMs has proven insufficient for addressing the domain-specific requirements of cybersecurity, as they generate fluent text but may hallucinate. To produce accurate predictions, they require additional contextual information that is not inherently available to them. [26] Retrieval-Augmented Generation (RAG) helps this by retrieving relevant documents and providing them as context to the model.

Why External Knowledge Grounding Matters

External knowledge grounding ensures that every claim, every affirmation, made by the LLM, can be traced back to a specific document chunk (from its own knowledge base). This provides some guaranteed advantages that are lacking from basic LLM interactions. We can focus on traceability, where the analyst can inspect the source documents that informed the response, enabling independent verification. Also, when adding new information that was changed, we can just do it quickly, by finding the specific changed chunk. Then we can focus on freshness**.** The knowledge base can be updated incrementally. Adding a new threat intelligence report takes seconds and it does not need any model retraining nor expensive resources. We should also highlight scope control. The retrieval step constrains the LLM to answer only from documents that exist in the knowledge base, reducing out-of-scope dialogue.

The two principal approaches to injecting domain knowledge into an LLM are fine-tuning and Retrieval-Augmented Generation (RAG). Fine-tuning literally bakes knowledge into the model weights through additional training on domain-specific data. RAG retrieves relevant documents at inference time and injects them into the prompt context. The trade-offs are substantial:

Table 3.3.1. Fine-tuning vs. RAG trade-off analysis.

Dimension	Fine-Tuning	RAG
Knowledge update	Full retraining required (hours–days)	Add document to index (seconds–minutes)
Knowledge freshness	Frozen at training cutoff	Current as of last index update
Traceability	Opaque weight encoding	Explicit source citations per answer
Cost	High (GPU compute for training)	Low (embedding + vector search)
Hallucination risk	Reduced but not eliminated	Reduced + grounded in sources
Specialisation depth	Deep (weights encode nuance)	Shallow (depends on retrieval quality)
Privacy	Training data ingested by model provider	Documents stay in local vector DB
Multi-domain coverage	One model per domain	One retriever, many knowledge bases

In the context of this project, RAG successfully serves as a threat intelligence copilot that maps SIEM alerts to known attack descriptions, providing SOC analysts with instant contextual explanations without requiring manual research.

Indexing Pipeline

The indexing pipeline transforms raw threat intelligence documents into searchable vector representations stored in Qdrant. The pipeline is triggered manually or on a schedule when new documents are added to the knowledge base directory. An agentic workflow powered by LLM is making sure the whole pipeline updates periodically to the newest threats that are indexed.

- Document ingestion. Source documents (PDF threat reports, MITRE ATT&CK technique descriptions, CVE summaries, Wazuh rule documentation) are loaded from the threat_intel/ directory. PDFs are parsed with pdfplumber; plain text and Markdown are read directly. Special care is taken for tables, unstructured data or different file formats.
- Chunking. Each document is split into overlapping fixed-size chunks (512 tokens, 128-token overlap). Chunk boundaries are passed through sentence endings where possible. Metadata (source filename, document title, page number, chunk index, ingestion timestamp) is attached to each chunk.
- Embedding generation. Each chunk is embedded into a 384-dimensional dense vector using the all-MiniLM-L6-v2 sentence-transformer model running locally. A BM25 sparse representation is also computed for hybrid retrieval.
- Vector storage. Dense vectors and sparse BM25 representations are stored in Qdrant collections with persistent volume (wazuh_qdrant_storage). The index is built once and survives container restarts.

Query Pipeline

At query time, the system performs hybrid retrieval combining dense semantic search and sparse keyword search, fuses the ranked results using Reciprocal Rank Fusion, and assembles a foundered prompt for the local LLM.

- Query embedding. The analyst's natural-language query is embedded using the same MiniLM model used at indexing time, producing a 384-dim query vector.
- Hybrid retrieval. Dense cosine similarity search and BM25 sparse search are run in parallel against the Qdrant collection. Each returns a ranked list of candidate chunks.
- RRF fusion. Reciprocal Rank Fusion combines the two ranked lists into a single merged ranking, rewarding chunks that rank highly in both dense and sparse retrieval. This technique was prefer over adding a cross-encoder.
- Top-k selection. The top k=5 chunks are selected from the fused ranking.
- Prompt assembly. Retrieved chunks are arranged with source citations and injected into a structured prompt template that the LLM will use. The LLM is instructed to answer only from the provided context (as seen in the system prompt from backend/server.py)
- LLM inference. The assembled prompt is sent to the local Ollama llama3.2 instance. The response, together with source tags, is returned to the analyst via the Flask API and rendered in the Chat UI.

Document Processing and Chunking Strategy

Chunking is the single most impactful design decision in a RAG system. The chunk is the atomic unit of retrieval: if the answer to a query is split across two chunks at a bad boundary, neither chunk will rank highly enough to be retrieved. Then the LLM will then hallucinate. Conversely, chunks that are too large consume context window budget without improving answer quality and dilute the semantic signal used for retrieval. Academic RAG literature, that form the basis of this type of systems, consistently identifies chunking as the one most dominant source of variance in end-to-end system performance. [27]

I. Fixed-Size vs. Semantic Chunking

Two primary chunking paradigms exist:

- Fixed-size chunking splits documents at a fixed token count, optionally with overlap. It is simple, deterministic, and fast. It might be risky when there is a big paragraph that expresses one complete concept, as it may be split mid-sentence.
- Semantic chunking splits at natural boundaries (paragraphs, sections, sentences) identified by structural or embedding-based coherence signals. It produces more semantically coherent chunks but requires more complex preprocessing and, therefore, produces variable-size chunks that may stress context windows on irregular basis.

For this system, fixed-size chunking with sentence-boundary alignment was selected. Pure semantic chunking was evaluated but rejected because the primary source documents (online attack frameworks, PDF threat reports, Wazuh rule documentation) have inconsistent structural markup. They make the selection of the boundaries difficult without manual annotation. The sentence-boundary alignment, applied to fixed-size chunks, gets most of the coherence benefit at low engineering cost.

II. Chunk Size Selection: 512 Tokens

Chunk size was selected through systematic evaluation over the range [128, 256, 512, 1024] tokens. The evaluation metric was Recall@5 on the held-out evaluation dataset.

Table 3.3.2. Chunk size ablation — Recall@5, MRR, and Hit Rate on the evaluation dataset.

Chunk Size (tokens)	Recall@5	MRR	Hit Rate	Avg Chunks/Doc	Notes
128	38.2%	0.521	64.3%	~82	High precision per chunk, too fragmented for multi-sentence answers
256	44.7%	0.631	74.1%	~41	Good balance for short factual queries
512	48.4%	0.695	82.9%	~21	Best Recall@5 and

					Hit Rate — selected
1024	45.1%	0.648	79.5%	~11	Context too broad; embedding signal diluted

Therefore, 512-token chunks outperform both smaller and larger alternatives. The 128-token chunks fragment multi-sentence concepts (e.g., a MITRE ATT&CK technique description that spans three paragraphs becomes 6+ fragments, none of which is complete). The 1024-token chunks embed too many distinct concepts into a single vector, making the semantic specificity way too big and not allowing it to be properly understood.

III. Overlap: 128 Tokens (25%)

A 128-token overlap (25% of chunk size) is applied between adjacent chunks. Overlapping has the same importance as choosing the right chunk size. This is because it serves as a sliding window that prevents information loss at different chunk edges. For example, a sentence that ends at token 512 and continues at token 513 is preserved intact in at least one of the two chunks.

The academic justification for overlap is clear: without it, the probability that a query answer falls exactly within a single chunk boundary is lower than 1. With 25% overlap, any 384-token passage appears in full in at least one chunk. Increasing overlap beyond 25% proves to be worthless in this thesis, as it yields poor performance and increases index size proportionally.

Metadata Enrichment

Each chunk is stored with the following metadata fields in the Qdrant payload. It was used to enable filtered retrieval and source citation:

Table 3.3.3. Metadata fields stored per chunk in Qdrant.

Metadata Field	Type	Purpose
source_file	string	Original document filename for citation
doc_title	string	Human-readable document title
page_number	int	Page within the source PDF
chunk_index	int	Position within the document (enables proximity ranking)
ingestion_timestamp	ISO datetime	Enables freshness scoring and stale-document filtering
category	string	Threat intel category (APT, C2, lateral-movement, etc.) for filtered search
word_count	int	Chunk length in words for context-budget planning

Embedding Fundamentals

An embedding model maps a text sequence into a dense vector in a high-dimensional space such that semantically similar texts produce vectors with high cosine similarity. Formally, for texts A and B with embedding vectors vA and vB:

$$\cos(\theta) = \frac{vA \cdot vB}{\|vA\| \cdot \|vB\|} \in [-1, 1]$$

Equation (3.6)

A cosine similarity of 1.0 indicates identical semantic content; 0.0 indicates orthogonality (no shared semantics); −1.0 indicates opposite meaning. At retrieval time, the query vector is compared against all stored chunk vectors, and the chunks with the highest cosine similarity to the query are returned.

The choice of embedding model determines the dimensionality of the vector space, the quality of semantic capture, inference latency, and whether the model runs locally or requires an external API.

Candidate Model Comparison

Table 3.2.4. Embedding model comparison. MTEB = Massive Text Embedding Benchmark average score.

Model	Dims	Params	MTEB Score	Latency (CPU)	Local	Notes
all-MiniLM-L6-v2	384	22M	56.3	~8ms	Yes	Selected — best latency/quality for local CPU
all-mpnet-base-v2	768	109M	63.3	~35ms	Yes	Higher quality, 4× slower on CPU
BGE-large-en-v1.5	1024	335M	64.2	~90ms	Yes	SOTA quality, impractical on CPU
E5-large-v2	1024	335M	62.3	~85ms	Yes	Strong but same CPU constraint as BGE
Instructor-XL	768	1.5B	61.1	~220ms	Yes	Task-instruction guided, very slow CPU
OpenAI text-embedding-3-small	1536	N/A	62.3	~120ms API	No	High quality, requires external API call
OpenAI text-embedding-3-large	3072	N/A	64.6	~180ms API	No	Best quality, privacy-incompatible for SIEM

When you query a RAG system, the request is converted into a high dimensional vector. To get accurate answers, the system must compare the query vector against millions of vectors of stored as Threat Intelligence documents, therefore using the principle of Approximate Nearest Neighbor. Many recent studies, including Sentence-BERT: Sentence Embeddings [28], contrast the traditional cyber defense methods, showing that the primary advantage of employing artificial neural networks (ANNs) is in their capacity for learning. Historically, patterns delineating normal and abnormal network activity were manually created by security professionals based on their expertise; however, we can now automatically discover these patterns using historical data within the network. In this project, the ANN algorithm chosen is Hierarchical Navigable Small World.

PostgreSQL as the Audit Helper

PostgreSQL (relational database) is used as the backup layer of the system, maintaining relational records of threat intelligence ingested from the external sources. While Qdrant is

useful for semantic similarity, PostgreSQL enables SQL queries for auditability: for example, we can filter alerts by time window, audit faster which CVEs were ingested. Because it persists each record's complete original payload as JSONB, it creates real durability and it holds what the vector store cannot generate. It works independently, being auditable and putting resilience to model change.

It's second functionality consists of being used by the CVE ingestion agent, with ingestion_log function, so the agent can write rows for how many CVEs got indexed, dropped, queued and evaluated. PostgreSQL has the relational capability that the vector store fundamentally lacks: every CVE decision is traceable to the run that produced it, and aggregate analytics (for example, to count the new indexed or dropped episodes per day) are expressible as a single join. Storing the complete source record as JSONB additionally enables controlled, vector-store-independent reloading of the knowledge base, so we can query the knowledge base also with SQL commands.

Additionally, it is used to create governance, as Qdrant contains only intelligence that was either automatically accepted at high confidence or explicitly approved by a human. Medium-confidence items are quarantined in PostgreSQL and never reach retrieval until properly stated so by a human. The vector store contains only verified intelligence, while PostgreSQL absorbs the uncertainty.

It contains 5 tables: alert_episodes (to keep the RAG corpus source format), cve_decisions (add/queue/drop decisions), ingestion_log (logs of the scheduled agent run), threat_intel (for the threat-intelligence records) and wazuh_alerts. It also has a view of pending_review for CVEs held out of the RAG for manual review.

4. Solution Implementation

While the previous chapter established why each method was chosen, this chapter describes how they are assembled into a working system. Therefore, it moves from the conceptual algorithms to a concrete, deployed architecture, detailing the engineering decisions, data flows, and implementation trade-offs. They turn the anomaly-detection ensemble model and the retrieval pipeline into a real, operational threat-detection engine, on top of Wazuh.

4.1. Technologies Used

Wazuh v.4.14.1 was used as the foundation of the project, to collect and normalize host telemetry and create JSON alerts stored in var/ossec/logs/alerts/alerts.json. Those alerts consist the raw material that the ensemble was fed with.

Considering the web service and dashboard layer, the following technologies were used:

- Flask, which is a lightweight Python web framework that exposes the system's HTTP API (anomaly scoring, the `/api/chat` RAG endpoint) and serves the dashboard, running on port 5000. It also includes Flask-CORS, that manages access for the browser front end, and Flask-Limiter, that applies request rate limiting to protect the API, a relevant concern for modern RAG systems.
- python-dotenv that loads configuration and secrets from a `.env` file.
- requests - the HTTP client used to call the local Ollama inference endpoint and to fetch external threat-intelligence websites in the automatic workflow (NVD, CISA-KEV).
- Front end represents a dark-themed HTML/JavaScript dashboard and chat interface (`frontend/chat.html`) rendering Markdown responses with source attribution.

For the RAG pipeline I used:

- Ollama (v0.16.3) with the Llama 3.2 model, that runs the Large Language Model entirely locally for answer generation (considering that the data contains confidential system information). Running this model locally helps in avoiding the exposal of security data to third-party APIs and removes per-token cost.
- Qdrant as the vector database that stores the threat-intelligence corpus. It supports hybrid retrieval, which outperformed a pure-dense FAISS index for this cybersecurity domain (initially tested).
- sentence-transformers (`all-MiniLM-L6-v2`) that generates the dense semantic embeddings indexed in Qdrant.
- fastembed (`Qdrant/bm25`) used to generate the BM25 sparse vectors that provide half of the hybrid search (to search lexically)

For the machine learning stack, the following where used:

- scikit-learn, provides the IF algorithm, together with the `StandardScaler` used to normalise features and the metric utilities used during evaluation (precision, recall, F1, confusion matrix).
- NumPy for all feature vectors and score calculations.
- Pandas is used for data wrangling during training-set assembly and evaluation reporting.

- joblib is used to serialise the trained model bundle (model, scaler, calibration parameters, and anomaly threshold) into a single `pkl` artifact. The pkl allows the detector to be loaded once and reused across the Flask server and the real-time alert monitor (so we don't train the model every time).
- Autoencoder (neural network) a second detector trained, which is paired with Isolation Forest to improve adversarial resistance.

For runtime environment and tooling:

- Python 3.13 is the implementation language for all back-end components (models, RAG pipeline, server, agents), isolated in a project venv.
- Docker / Docker Compose for containerises to orchestrate the services (Qdrant and PostgreSQL) with health checks and named persistent volumes.
- Linux (WSL2, kernel 6.6) for the development and runtime host, mirroring a production Linux server deployment.
- Git for version control for the entire codebase.

4.2. Ensemble Implementation

This section describes how the ensemble detection pipeline is deployed in practice, focusing on the real-time alert monitoring mechanism and the controlled attack simulation used to validate it. The objective is to demonstrate that the system can ingest live security events, process them through the ML ensemble, and surface enriched alerts to the analyst.

Alert Monitoring and Attack Simulation

The monitoring session for alerts.json starts from the end of the file to avoid loading the long history into memory; only newly appended lines are collected. This is a design decision for a 'live' dashboard, with low I/O and memory cost. Upon initialization, it creates the necessary directory structure for model storage and enhanced alert output, then instantiates the AI Threat Engine with configured paths for the ML ensemble model, vector database, and LLM API endpoint. The scope of this was to ensure that the user will receive only new alerts from the current session.

Testing includes collecting a test set after training, simulation scripts, and evaluation reports (confusion matrix, score distributions, per-rule interpretation). All the evaluations were stored in data/ so they can be further analysed, while also the scripts can be reused to compare results.

The project includes an attack simulation framework (attack_simulation/simulate_attack_for_wazuh.sh) that generates controlled security events to test the detection pipeline. By running this, the real Wazuh deployment captured authentic security events including trojanized file detections (signatures matching /bin/passwd, /usr/bin/chfn, /usr/bin/chsh), kernel-level rootkit anomalies (hidden files in /tmp), multiple failed authentication sequences (PAM failures, unix_chkpwd failures, FAILED SU attempts), listening port changes (Docker proxy ports, Wazuh service ports), and MITRE ATT&CK-

tagged events (T1548.003 sudo Caching, T1078 Valid Accounts, T1110.001 Password Guessing, T1014 Rootkit).

Training Data Collection and Strategy

The project code embrace the strategy of aggregating alerts into combined files, setting up daily alert collection pipelines, separating 'clean' alerts from those used for attack evaluation, and keeping a distinct test file (all_test_alerts.json) for measurements and evaluations after retraining. The training strategy of splitting the data works as follows: the training set is split in the benign subset that was used exclusively to fit the ensemble. Because it is a one-class classifier, no attack examples are used during training.

User-defined exceptions in the backend (benign_rules.json) impact both training and inference: a rule marked benign is no longer handled as an attack in building the ground truth for reports, and the score can be capped to avoid frequent false alarms.

The effectiveness of the Isolation Forest and Autoencoder ensemble model rely critically on the quality and representativeness of the training data. In this project, training data was collected from two complementary sources:

Source 1: Daily Wazuh Alert Collection. I've built a custom data pipeline (collect_training_data.py) to periodically accumulate real Wazuh alerts over time. The script is designed to run daily, either manually or via cron scheduling, and performs the following steps: (a) reads the live Wazuh alerts file at /var/ossec/logs/alerts/alerts.json, (b) filters alerts by date, (c) deduplicates using a fingerprint made of timestamp, rule ID, agent ID, and message prefix, (d) saves daily snapshots to a structured directory (data/training/daily_logs/YYYY-MM-DD.json), and (e) merges all daily files into a single combined training file (data/training/combined/all_alerts.json). This operational data captures the normal baseline behavior of the monitored system, including authentication events, system checks, rootcheck results, and network monitoring data.

Source 2: External Attack Data from WordPress Plugin. To train the model with genuine malicious patterns, I further collected data from a WordPress security plugin deployed on a website that was subjected to real-world attacks over a two-week period. This external dataset included indicators of brute-force login attempts, SQL injection probes, cross-site scripting (XSS) payloads, and automated vulnerability scanning, providing the model with concrete examples of attack behavior that complement the operational baseline from the local Wazuh deployment.

The combination of these two data sources is strategically important: the local Wazuh data teaches the model what "normal" looks like in the specific deployment environment, while the external attack data ensures the model has been exposed to real-time malicious patterns. The training pipeline prioritizes data sources in order: (1) combined collected training data, (2) live Wazuh alerts, (3) local sample alerts for development and testing.

Initial Evaluation (Baseline Model) and Retraining — Results

The reported experiment measures performance on two sets: (D_train) combined training data with approximately 2200 alerts, comprising 1,534 benign and 637 attack-labelled

alert; (D_test) test set with 182 alerts, 171 benign alerts and 11 attack alerts. Labeling follows the same is_attack_alert function as in the project code, with user benign exceptions disabled in the reproducible local experiment (empty set), for consistency with the automatically generated report.

The initial baseline model was trained using a supervised strategy, where only benign alerts were passed to the Isolation Forest, with the anomaly threshold calibrated as the 90th percentile of clean scores. However, the intention of the thesis is to analyse an unsupervised environment, that is autonomous and has various data. Therefore, the retrained model adopts a fully unsupervised strategy: the Isolation Forest is fitted on all 2,171 alerts without any label filter (contamination='auto'); the threshold comes from a natural score-gap heuristic applied to the full score distribution.

Table 4.2.1. Isolation Forest evaluation results — training set (D_train).

Model	Score Sep.	TPR%	FPR%	Prec%	Rec%	F1%	Threshold
Baseline	75.58	95.69	8.83	83.74	95.69	89.32	49
Retrained	47.8	90.3	6.2	85.8	90.3	88.0	40

Table 4.2.2. Isolation Forest evaluation results — test set (D_test).

Model	Score Sep.	TPR%	FPR%	Prec%	Rec%	F1%	Threshold
Baseline	56.13	81.82	4.09	56.25	81.82	66.67	49
Retrained	44.9	72.7	2.9	61.5	72.7	66.7	40

On the D_train set, we can observe the structural consequence of the changed training strategy. Because attack-labelled alerts are now included in the training data, the Isolation Forest cannot isolate them as cleanly as when it was trained on benign-only data. However, the new model doesn’t have the unfair advantage so it needs to discover structure entirely on its own. Despite this, the model achieves a false positive rate of 6.2% (vs. 8.83% for the baseline) and slightly higher precision (85.8% vs. 83.7%), at the cost of a moderate recall decrease (90.3% vs. 95.7%). F1-score remains comparable (88.0% vs. 89.3%).

On the D_test, the pattern is consistent. The FPR improves (2.9% vs. 4.09%) and precision increases (61.5% vs. 56.3%), while recall decreases (72.7% vs. 81.82%). The F1-score is virtually unchanged (66.7% vs. 66.7%). The persistent weakness is single failed login events (PAM: User login failed, 48 alerts, 14% detection on D_train), whose feature profiles overlap heavily with benign typos.

The practical interpretation is that the unsupervised model trades a portion of recall for a measurable reduction in false positives and does not require any labelled data during training. The small size of the attack class in D_test (11 alerts) limits the statistical significance of test-set comparisons; the D_train figures on 637 attack alerts provide a more reliable estimate of generalisation.

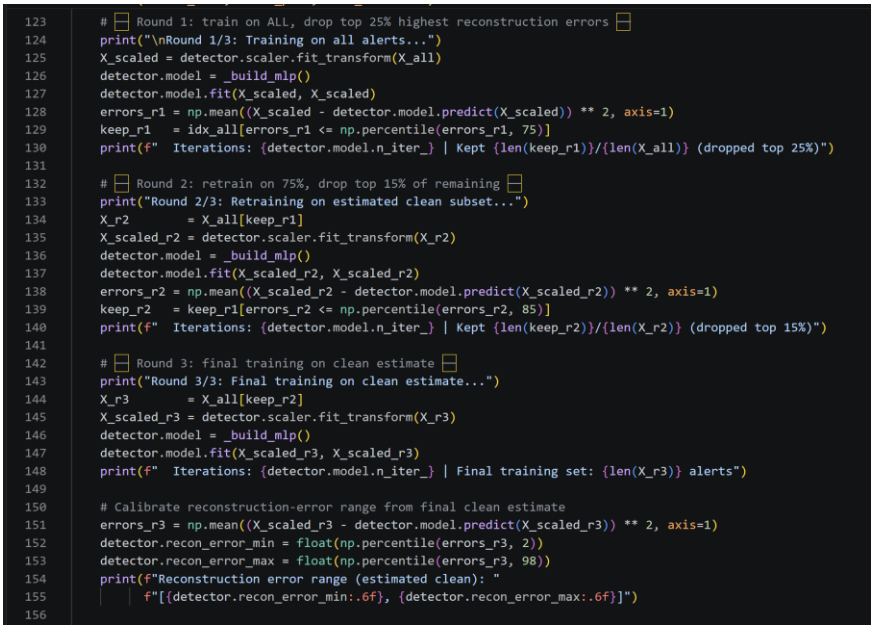
To justify the choice, the baseline recall was inflated by a training shortcut that is not reproducible in production. This approach of labelling the attacks will not be able to spot new attacks in a production environment. Training exclusively on “clean” data assumes a labelling capability that does not exist in production systems. In addition, the autoencoder already forces the ensemble to be unsupervised, and the FPR improvement matters more than the recall loss in a real analyst workflow.

Iterative trimming loop for Autoencoder

The autoencoder was trained on completely unsupervised traffic, leaving intentionally 30% attack alerts mixed in normal traffic in the training data. This design decision reflects the operational reality of SIEM deployments: in production environments, security analysts cannot guarantee that any historical alert log is free of attack activity. Maintaining a verified clean training corpus requires continuous manual labelling, which is labour-intensive, domain-expert-dependent, and fundamentally incompatible with the goal of an autonomous detection system. Training on contaminated data is made viable through iterative trimming, a self-supervised bootstrapping procedure. Therefore, if I did not include iterative trimming, the model would have learned to reconstruct attacks too.

The justification behind this method is that attacks reconstruct badly (high error) while clean alerts reconstruct well (low error). Therefore, I have implemented a strategy that drops the first 25% highest errors in the first round (to remove most of the attacks without knowing which ones they were) then refine, dropping again the top 15% errors in the second round. This approach catches attacks that slipped through round 1 because they were borderline.

The last time (round 3), it will be the last retraining, on the so called “cleaned” training set. Then calibrate the reconstruction error range from this final set so the scoring is normalized properly. No attack labels are required at any stage. The model bootstraps its own understanding of normality through iterative self-correction. One advantage of this approach is that it is self-correcting across retraining cycles: as new alerts are collected, the trimming procedure re-estimates the clean distribution from scratch, adapting to environmental drift without manual intervention.



```
    # ── Round 1: train on ALL, drop top 25% highest reconstruction errors ──
    print("\nRound 1/3: Training on all alerts...")
    X_scaled = detector.scaler.fit_transform(X_all)
    detector.model = _build_mlp()
    detector.model.fit(X_scaled, X_scaled)
    errors_r1 = np.mean((X_scaled - detector.model.predict(X_scaled)) ** 2, axis=1)
    keep_r1   = idx_all[errors_r1 <= np.percentile(errors_r1, 75)]
    print(f"  Iterations: {detector.model.n_iter_} | Kept {len(keep_r1)}/{len(X_all)} (dropped top 25%)")

    # ── Round 2: retrain on 75%, drop top 15% of remaining ──
    print("Round 2/3: Retraining on estimated clean subset...")
    X_r2        = X_all[keep_r1]
    X_scaled_r2 = detector.scaler.fit_transform(X_r2)
    detector.model = _build_mlp()
    detector.model.fit(X_scaled_r2, X_scaled_r2)
    errors_r2 = np.mean((X_scaled_r2 - detector.model.predict(X_scaled_r2)) ** 2, axis=1)
    keep_r2   = keep_r1[errors_r2 <= np.percentile(errors_r2, 85)]
    print(f"  Iterations: {detector.model.n_iter_} | Kept {len(keep_r2)}/{len(X_r2)} (dropped top 15%)")

    # ── Round 3: final training on clean estimate ──
    print("Round 3/3: Final training on clean estimate...")
    X_r3        = X_all[keep_r2]
    X_scaled_r3 = detector.scaler.fit_transform(X_r3)
    detector.model = _build_mlp()
    detector.model.fit(X_scaled_r3, X_scaled_r3)
    print(f"  Iterations: {detector.model.n_iter_} | Final training set: {len(X_r3)} alerts")

    # Calibrate reconstruction-error range from final clean estimate
    errors_r3 = np.mean((X_scaled_r3 - detector.model.predict(X_scaled_r3)) ** 2, axis=1)
    detector.recon_error_min = float(np.percentile(errors_r3, 2))
    detector.recon_error_max = float(np.percentile(errors_r3, 98))
    print(f"Reconstruction error range (estimated clean): "
          f"[{detector.recon_error_min:.6f}, {detector.recon_error_max:.6f}]")

```

Figure 4.2.3. Iterative trimming loop implementation.

Three-Model Comparison: IF vs AE vs Ensemble

Table 4.2.4. Side-by-side comparison of all three models on the May 2026 evaluation dataset.

Metric	Isolation Forest	Autoencoder	Ensemble
TP	614	584	614
FN	26	56	26
FP	122	~92	~152
TN	1,409	~1,439	~1,379
Precision	83.4%	~86.4%	~80.2%
Recall	95.9%	91.3%	95.9%
F1-Score	88.0%	~88.7%	~87.4%
False Positive Rate	8.0%	~6.0%	~9.9%
Score Separation	75.1 pts	~75.2 pts	75.1 pts
Unique contribution	30 extra attacks	Higher prec.	Tiered confidence

This is a comparison of the three models to observe operational trade-offs:

Isolation Forest achieves the best recall and competitive F1. It is the single most important component and would be the recommended choice if only one model can be deployed. Its weakness is false positives (8.0% FP rate) and no inherent confidence gradation. So, every flagged alert is equally 'anomalous' from the model's perspective.

Autoencoder achieves lower recall but higher precision and lower false positive rate (~6.0%). It is better at avoiding false alarms on unusual-but-benign alerts that the Isolation Forest flags. On its own it misses 30 more attacks than the IF. It has a particular sensitivity to the independent features that I correlated, which makes it a valuable second opinion.

The ensemble achieves the same recall as the Isolation Forest while adding tiered confidence information. The cost is a slightly higher FP rate because it is the union detector. The key advantage is not only the metric improvement but also the operational value: the analyst knows whether an alert was flagged by both models (CRITICAL, so act now) or only one (POSSIBLE/HIGH, so review when time allows). This reduces a lot of mental burden in environments with a really high volume of alerts, which is the primary reward of the ensemble beyond those raw metrics.

Tier Logic: CRITICAL / HIGH / POSSIBLE / NORMAL

The ensemble verdict is determined by the binary anomaly decisions of each model (is_anomaly_IF and is_anomaly_AE), not by their continuous scores. This is intentional: combining binary decisions preserves the calibrated threshold semantics of each model, rather than introducing new threshold assumptions in the combination layer.

Table 4.2.5. Ensemble tier logic based on per-model binary anomaly decisions.

is_anomaly_IF	is_anomaly_AE	Tier	Interpretation
True	True	CRITICAL	Both models agree - highest confidence anomaly
False	True	HIGH	Only Autoencoder flags - high precision catch
True	False	POSSIBLE	Only Isolation Forest flags - low-signal catch
False	False	NORMAL	Neither model flags - alert is benign

The tier names reflect the confidence hierarchy: CRITICAL represents consensus between two independent detection approaches. In the evaluation dataset, 91.3% of detected attacks fall in the CRITICAL tier, confirming that the two models overwhelmingly agree on clear attack patterns. The HIGH tier (only AE flags) and POSSIBLE tier (only IF flags) represent the margin cases where one model's sensitivity captures something the other misses.

All three anomaly tiers (CRITICAL, HIGH, POSSIBLE) result in is_anomaly = True and trigger the same downstream processing (enrichment, dashboard flagging, RAG context injection). The difference is surfaced to the analyst through the tier label, not through binary filtering.

Combined Score Formula

In addition to the tier label, the ensemble computes a combined score for ranking and prioritisation within each tier. The combined score is a weighted average of the two normalised

scores, with a slightly higher weight on the Autoencoder to reflect its higher precision in the standalone evaluation:

$$\text{Combined}_{\text{score}} = \text{clip}(\, 0.45 \times \text{score}_{\text{IF}} + 0.55 \times \text{score}_{\text{AE}}, 0, 100\,) \text{ [for CRITICAL and HIGH]}$$

Equation (4.1)

$$\text{Combined}_{\text{score}} = \text{clip}(\, (0.45 \times \text{score}_{\text{IF}} + 0.55 \times \text{score}_{\text{AE}}) \times 0.65, 0, 100\,) \text{ [for POSSIBLE]}$$

Equation (4.2)

$$\text{Combined}_{\text{score}} = \text{clip}(0.45 \times \text{score}_{\text{IF}} + 0.55 \times \text{score}_{\text{AE}}, 0, 100) \text{[for NORMAL (neither flags it } or \text{ AE unavailable edge case)]}$$

Equation (4.3)

The dampening factor of 0.65 applied to POSSIBLE tier alerts reduces the combined score to reflect the lower confidence of a single-model detection. This ensures that POSSIBLE alerts are ranked below CRITICAL and HIGH alerts in the dashboard, even if the Isolation Forest score alone is high. For NORMAL alerts, the combined score is computed without dampening and serves as a sub-threshold proximity indicator for borderline cases.

Evaluation

The models were trained in an unsupervised manner on unlabeled data. To quantify detection performance, ground truth labels were assigned post-hoc using a rule-based classifier (attack_labels.py) for evaluation purposes only.

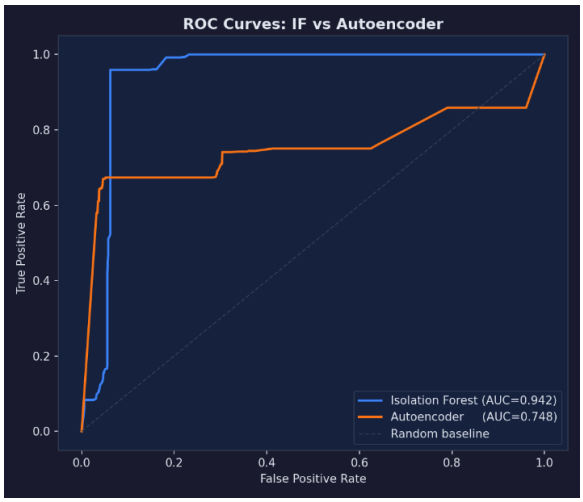


Figure 4.2.6. ROC-AUC curve for both ML models

The Isolation Forest achieves AUC = 0.942, substantially outperforming the Autoencoder (AUC = 0.748) as an independent, standalone detector. However, the models always show complementary error patterns therefore attacks missed by IF are recovered by the AE. This is motivating for the ensemble combination which achieves the highest recall of 93.6%.

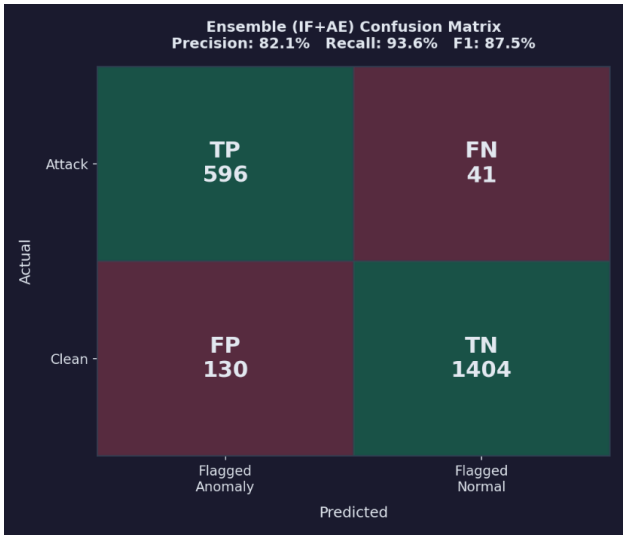


Figure 4.2.7. Confusion-matrix of the ensemble model

The ensemble has the fewest missed attacks (FN=41 vs 62 vs 208), which is the most important metric for a security system. Comparing the ensemble against the standalone Isolation Forest (Precision 83.4%, Recall 95.9%, F1 89.2%), the ensemble yields marginally lower metric scores. This is consistent with the design intent: the ensemble does not optimise for raw metric performance but for adversarial robustness. This is an accepted trade-off.

4.3. Retrieval-Augmented Generation

The system operates in two distinct phases: an offline indexing pipeline and an online query pipeline. These are described below and illustrated in the XML architecture diagram found in Appendix as Figure 4.

Table 4.3.1. Steps of the hybrid retrieval pipeline (dense + sparse + RRF).

Step	Description
S1	Building the episode text from summary, tags, and entities.
S2	Dense encoding with Sentence Transformer and normalization.
S3	Sparse encoding (BM25-like) for lexical matches.
S4	Prefetch candidates (e.g., 40) from each channel + metadata filters.
S5	RRF fusion and return of top-k results to the application layer.

The RAG architecture in this project follows a pipeline of four stages:

Stage 1 – Ingestion

Threat intelligence is ingested from multiple structured sources: MITRE ATT&CK technique descriptions (downloaded from the official STIX repository), vendor security

advisories (Microsoft Security, CISA), indicators of compromise (IOCs), YARA malware detection rules (from Yara-Rules/rules GitHub repository), Sigma detection rules (from SigmaHQ/sigma), and internal threat notes. Each source is stored in a dedicated directory under threat_intel/ with timestamped metadata and freshness scores.

A user can add his own documents, having enabled this option in a separate file of the Threat Engine. The user-uploaded documents get written in Postgres (as backup) then in Qdrant embedding vector. Manually curating the corpus does not scale and is error-prone. The natural remedy is to automate ingestion. Therefore, the system includes a planned agentic extension that periodically enriches the RAG knowledge base with newly published CVEs found on the Internet. A scheduled agent fetches vulnerability websites from NVD and CISA-KEV, then uses an LLM to score each CVE's relevance (0-10) against the deployment environment.

The agent is configured to be environment aware (since the system is a Linux-based Wazuh host), therefore it scores relevance, not severity, against this explicit deployment profile. So, the scheduled workflow fetches new CVEs on a cron schedule, deduplicate what is already present in the knowledge base (against the PostgreSQL record of accepted intelligence), boots its relevance and score it with an LLM. Then, it acts by routing the CVE according to its final score into 3 categories (indexed if it has the score above 7, queued if the score is between 4 and 6 and if it needs human review, or dropped, otherwise). Then it performs the corresponding action. Subsequently it logs the CVE decision row into PostgreSQL and closes the new rollup row.

The scheduled agent changes the RAG knowledge base from a static corpus into a continuously database, without altering the retrieval pipeline itself. By that, accepted CVEs are converted into the episode representation that can be used by the same Qdrant collection and indexer. If we from the assistant's perspective, they are identical from manually curated intelligence and become retrievable immediately after being added and indexed.

Stage 2 - Chunking and Episode Building

Raw Wazuh events are classified by semantic type (process, authentication, network, alert) and grouped into structured "security episodes" using specialized episode builders. Each episode is a self-contained security object with fields for: episode ID, type, time range, extracted entities (users, hosts, IPs, domains, hashes), source attribution, ATT&CK tags, a compact summary (200-2000 tokens), and raw event references. [2] This chunking strategy ensures that semantically related events are grouped together, improving retrieval precision.

Stage 3 - Embedding and Indexing

Episode summaries, entity strings, and tag sequences are encoded into a single text representation that serves as the input to two parallel embedding pipelines. Dense semantic embeddings are produced by the all-MiniLM-L6-v2 sentence-transformer model [29], which produces 384-dimensional embeddings optimized for semantic similarity search. It was selected for its optimal balance between embedding quality and inference speed, producing embeddings approximately 5x faster than larger models while retaining over 90% of the semantic similarity performance on standard benchmarks (STS Benchmark). [30]

In parallel, sparse BM25 keyword embeddings are computed using the **fastembed** library with the Qdrant/bm25 sparse model, which produces token-frequency weighted sparse vectors that capture exact and rare term matches —characteristics that dense embeddings systematically underweight.

Both vector types are persisted in **Qdrant** [31], an open-source vector database, across two named collections: wazuh_threat_intel for curated threat intelligence episodes and wazuh_alert_episodes for live alert episodes. Qdrant indexes dense vectors using the Hierarchical Navigable Small World (HNSW) algorithm, which provides sub-linear approximate nearest-neighbour query time at high recall. Sparse vectors are indexed using a dedicated inverted-index structure within Qdrant. Episode metadata fields — source_type, episode_type, and tags — are indexed as payload fields to enable low-latency pre-filtering prior to vector search. Raw episode records are additionally persisted to a PostgreSQL relational store to support structured metadata queries independent of vector similarity.

Stage 4- Hybrid Retrieval: BM25 + Vector Search

The retrieval component employs a hybrid strategy that combines two complementary scoring signals to maximize retrieval quality. Rather than a weighted sum, the two ranked candidate lists are merged using Reciprocal Rank Fusion (RRF), an algorithm that rewards documents ranking well in both search modes simultaneously.

Semantic Search: The query is encoded using the same all-MiniLM-L6-v2 model and compared against indexed episode embeddings using cosine similarity in Qdrant. This captures meaning-level similarity, enabling the system to match "SSH brute force attempt" with "Multiple failed authentications via Secure Shell" even without shared keywords.

Keyword Search (Sparse BM25): Sparse vectors are generated using the Qdrant/bm25 model via fastembed. BM25 scores documents based on term frequency, inverse document frequency, and document length normalization. This ensures exact technical terms, IP addresses, CVE identifiers, tool names, are matched precisely, complementing the semantic search.

RRF Fusion: Reciprocal Rank Fusion combines the results from two different search methods. First, each retriever independently finds its top 40 most relevant documents. This produces two ranked lists of candidates. Instead of choosing one list over the other, RRF merges them by giving each document a score based on its position in each ranking. Documents that appear near the top of a list receive more credit than those appearing lower down. The score is calculated by adding together small contributions from each ranking, with k=60 as smoothing parameter. Documents appearing well in both retrievers receive a larger combined score and rise to the top of the final results. The smoothing constant of k=60 prevents the very top positions from dominating too strongly, allowing documents that perform well across both retrieval methods to be also considered.

Dense embeddings transform episode text (summary + labels + entities) into a vector space where proximity signifies semantic similarity. Storing and searching these vectors in Qdrant enables scaling to hundreds of thousands/millions of points, with metadata filtering (source_type, tags, episode_type).

Therefore, the system uses two representations: a dense vector (cosine similarity) and a sparse vector associated with a BM25-like model via fastembed, followed by fusion of candidate lists through Reciprocal Rank Fusion (RRF).

Stage 5 - RRF instead of Cross-Encoder re-ranking

Cross-encoder re-ranking was evaluated but removed. The ms-marco-MiniLM-L-6-v2 model was not suited to the cybersecurity domain, it degraded retrieval quality on security-specific queries. Sufficient precision is achieved through the large prefetch pool (40 candidates per retriever) and RRF fusion. Cross-encoder re-ranking remains a viable extension for deployments with a domain-fine-tuned model.

This core problem arrived because cross-encoders are very sensitive to domain. A model trained on web search has no understanding that rule.id 5710 is more relevant to an SSH attack query than a generic authentication document. RRF doesn't have this problem because it makes no learned judgement, it just trusts rank position from two independent retrievers.

Stage 6 - Retrieval and LLM Synthesis

When a new alert arrives, the system generates a query from the event description and searches the knowledge base using hybrid retrieval. The LLM (configured with Ollama/llama3.2 or OpenAI-compatible API) receives the alert context alongside the retrieved threat intelligence and generates a structured analysis including threat level assessment, confidence score, pattern identification, and actionable recommendations. The LLM operates under specific rules: it must respond in JSON format with predefined fields, use a low temperature (0.3) for deterministic outputs, and base its analysis on the similarity-ranked retrieved context. This low temperature is recommended in this sort of environments to produce a good mix of creativity and high determinism.

Top-k Selection: k=5

The number of chunks that are retrieved to be exactly k is used to control the precision-recall trade-off in the context stage. Increasing k to this number (5) improves the chance that the relevant chunk is retrieved (recall) but also increases the context window consumed (the text that model can “see”). It risks weakening the prompt with irrelevant content (precision).

Table 4.3.2. Impact of top-k on retrieval and generation quality.

k	Hit Rate	MRR	Recall@k	Context tokens	LLM faithfulness
1	51.2%	0.512	18.3%	~51	High (single focused source)
3	71.4%	0.648	36.7%	~1,536	Good
5	82.9%	0.695	48.4%	~2,560	Good — selected
10	89.3%	0.701	58.1%	~5,120	Reduced (context dilution)
20	91.1%	0.694	63.2%	~10,240	Poor (exceeds llama3.2 context)

k=5 was selected as it maximises Hit Rate within the context budget of the local llama3.2 model (8K context window). It feels intuitive that retrieving more documents gives the model more information to work with. But past a certain k, each added chunk dilutes the others, proving that it does not mean better answers.

LLMs distribute attention unevenly across long contexts, a phenomenon documented as the lost-in-the-middle effect. When many chunks are all added into a single prompt, the model focuses strongly on content near the beginning and end of the context, while chunks in the middle receive proportionally weaker attention, regardless of their relevance. At k=10 or more, retrieved chunks begin competing for the model's attention, as chunks might land in the middle of a very long context and therefore, value differently. The model cannot focus equally on all of them, so, it loses faithfulness.

Prompt Engineering and Context Assembly

The local llama3.2 model has an 8,192-token context window. This budget must accommodate: the system prompt (~200 tokens), the retrieved context (~2,560 tokens for k=5 at 512 tokens each), the source citation headers (~100 tokens), the user query (~50–200 tokens), and the LLM's response (~300–600 tokens). Total consumption is approximately 3,200–3,700 tokens, comfortably within the 8K limit for k=5.

The system prompt is structured to heavily force grounding and suppress hallucination, the primary mission of RAG. The template follows the instruction hierarchy: system identity → grounding constraint → citation instruction → retrieved context → user query.

```
74
75  SYSTEM_PROMPT = """You are a cybersecurity analyst assistant powered by the Wazuh AI Threat Engine.
76  You have access to two data sources:
77  1. A threat intelligence knowledge base containing MITRE ATT&CK techniques (823), YARA detection rules (378), and vendor security advisories (132).
78  2. Real Wazuh alert logs from this system — actual security events that have been detected.
79
80  When answering questions:
81  - If the user asks about what happened on the system, attacks detected, or specific alerts, use the Wazuh alert context.
82  - If the user asks about techniques, detection methods, or threat intelligence, use the knowledge base context.
83  - Cite specific technique IDs (e.g., T1055), rule IDs, alert timestamps, and severity levels.
84  - Summarize patterns you see in the alerts (e.g., repeated brute force, privilege escalation chains).
85  - Provide actionable security guidance based on what's observed.
86  - Keep answers concise but thorough.
87
88  IMPORTANT — Distinguish normal activity from threats:
89  - Alerts with rule level 1–5 and Anomaly Score labeled NORMAL are almost certainly routine system activity (logins, sudo, PAM sessions, cron jobs).
90    Do NOT map these to attack techniques. State clearly that the activity appears benign.
91  - Only escalate to threat analysis when: rule level >= 7, OR Anomaly Score is HIGH, OR there is a clear pattern of malicious intent
92    (e.g., repeated failures, off-hours, known-bad IPs).
93  - When threat intel context is absent (low-level alerts), say so explicitly and do not invent threat connections."""
94
```

Figure 4.3.3. System prompt design

This thesis also wants to address anti-hallucination grounding by giving instructions to the model. Three specific instructions in the system prompt combat different hallucination modes:

- **Answer must be ONLY from context.** Prohibits the LLM from supplementing retrieved context with parametric (training-time) knowledge. Without this constraint, LLMs frequently blend retrieved facts with memorised facts, producing answers that are partially correct and partially hallucinated.
- **Explicit out-of-scope response.** Instructing the model to say 'I do not have information about this in the current knowledge base' when the answer is absent is critical. Without it, the model defaults to generating a plausible-sounding answer from parametric knowledge, which is indistinguishable from a correct answer to the analyst.

- **Source citation requirement.** Requiring [Source: ...] tags after each claim creates a verifiable audit trail and forces the model to associate each fact with a specific retrieved chunk, reducing unsourced confabulation.

System Metrics

- **End-to-end latency.** Time from query submission to first response token (ms).
- **Retrieval latency.** Time for Qdrant hybrid search + RRF (ms).
- **Embedding latency.** Time for query embedding (ms).
- **LLM time-to-first-token.** Ollama inference latency (ms).
- **Token cost.** Total prompt + completion tokens per query (relevant for API-based deployments).

Evaluation Dataset

The evaluation dataset contains 82 query-chunk pairs across 6 threat intelligence categories, constructed by human annotation of the knowledge base documents. Initially, a smaller dataset was used having only Wazuh alerts but a more detailed, per category set is statistically more meaningful.

Table 4.3.4. Evaluation dataset composition.

Category	Queries	Avg Relevant Chunks	Notes
MITRE ATT&CK techniques	22	2.1	Technique descriptions, sub-techniques, mitigations
CVE / vulnerability details	18	1.8	Specific CVE descriptions and affected software
Lateral movement TTPs	12	2.4	Movement between hosts, credential reuse
C2 / exfiltration patterns	14	2.2	Command-and-control infrastructure patterns
APT group profiles	9	3.1	Nation-state actor TTPs and campaign history
Wazuh rules and responses	7	1.5	Platform-specific rule interpretation

Retrieval Performance by Category

Retrieval was evaluated with the deployed hybrid configuration (dense MiniLM embeddings + BM25 sparse, fused by Reciprocal Rank Fusion) over all 82 queries. Find the results of the retrieval evaluation in Appendix as Table 2 for the MITRE attacks then Table 3 for the retrieval evaluation per category.

Table 2 shows that genuinely weak spots are credential-access (40%), and privilege-escalation (50%), as seen in the 8 missed queries, all of which want YARA/technique IDs but retrieve group entries instead. Instead, Table 3 highlights the fact that retrieval works well overall, but quality depends a lot on the category. CVEs and APT group profiles are basically

perfect, the system finds the right source every time and ranks it at or near the top. Lateral movement and C2/exfiltration are also solid.

Retrieval failures is the reason why incorrect generated answers exist. Through, it is necessary for us to manually inspect the 17.1% of queries where no relevant chunk appeared in top-5, we can justify the low performance of the retrieval and why it failed based on these factors:

- Semantic ambiguity. Queries using colloquial or abbreviated terminology (e.g., 'beaconing' for C2 heartbeat communication) did not match the embedding space of the threat reports. They are written to be formal, in specific terms, using phrases like 'command-and-control callback' or 'periodic HTTP request'. BM25 was equally unable to match this, because no keyword overlap existed.
- Hard to reason when there are many documents. Some queries require information from two or more documents (e.g., 'how does T1078 relate to lateral movement?'). Neither document individually answers the question, so, answering requires combining facts from both. RAG with independent chunk retrieval fundamentally cannot handle this case.
- C2 and APT category weakness. The two weakest categories share a common characteristic: their documents use highly specific and narrow vocabulary that does not generalise well in the embedding space. A query about 'Cozy Bear' infrastructure does not semantically retrieve a document about 'APT29 C2 patterns' because the embedding model has not been trained to associate these aliases.

Generation Quality

While the retrieval evaluation measures whether relevant evidence is taken out, it does not assess the quality of the natural-language answers in the generation stage. This section defines the solution used to evaluate three properties of the generated responses: faithfulness, citation correctness, and refusal behaviour. All three are reference-free properties: they are judged against the retrieved context and the system's own citations rather than against gold answers. They are evaluated by structured human annotation, with an automated cross-check as LLM-as-judge.

The evaluation set consists of the same 82 queries used for retrieval, for which the deployed pipeline (hybrid RRF retrieval at k = 5 followed by generation with the local llama3.2 model) produces an answer and an associated list of cited sources. Refusal behaviour is evaluated on a separate, purpose-built set of unanswerable queries, described below.

Faithfulness means that every factual claim in the answer is supported by the retrieved chunks. This catches hallucination without the model adding facts that aren't in the context. It is the primary protection against hallucination, for the model to introduce facts absent from the evidence. The method that was used was to adopt the claim-decomposition approach used in the RAGAS framework. Each answer is decomposed into a set of small claims, minimal,

independently verifiable but true statements. Each claim is then checked against the k retrieved chunks and labelled supported if its content is entailed by at least one chunk, or unsupported otherwise. This should be done by a human, but in its absence, LLM can solve it (as shown in the solution).

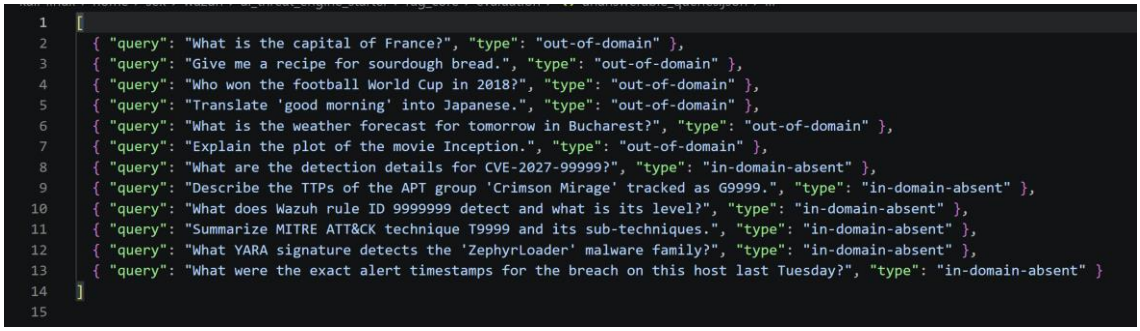
Next, we are concerned about citation correctness. The system emits explicit inline citation markers ([i]) that attribute statements to specific retrieved sources. Citation correctness measures whether these attributions are accurate. Because the pipeline is citation-native, attribution can be measured directly, which is not possible for systems that produce unattributed text (other LLMs that hallucinate). Two complementary sub-metrics are distinguished:

- Citation precision — of the citations the answer makes, the fraction whose cited source actually supports the attached statement. To check if the chunk is wrong.
- Citation recall (coverage) — of the statements that require attribution, the fraction that carry a citation. This captures uncited assertions.

Citation markers are parsed from each answer and mapped to the corresponding entries in the system's source list. For each citation, the cited chunk and the sentence it is attached to are compared, and the citation is labelled correct if the chunk supports the sentence.

Refusal behaviour (out-of-scope handling) measures whether the system correctly declines to answer queries that cannot be answered from the knowledge base, rather than inventing a response. This directly tests the system prompt's instruction (discussed above) to state explicitly when no relevant context is available and to avoid inventing connections. So, refusal can only be measured on queries that should be refused. A dedicated set of unanswerable queries was created, with questions having nothing to do with the knowledge base and they were separated from the query evaluation set. It comprises two types:

- Out-of-domain queries, wholly unrelated to threat intelligence (like general-knowledge questions), for which the knowledge base contains nothing relevant; and
- In-domain-but-absent queries, plausible security questions whose answers are not present in the indexed corpus. These are the more demanding cases, as retrieval still returns superficially related but irrelevant chunks, testing whether the system refuses despite being given distracting context.



```
[
  { "query": "What is the capital of France?", "type": "out-of-domain" },
  { "query": "Give me a recipe for sourdough bread.", "type": "out-of-domain" },
  { "query": "Who won the football World Cup in 2018?", "type": "out-of-domain" },
  { "query": "Translate 'good morning' into Japanese.", "type": "out-of-domain" },
  { "query": "What is the weather forecast for tomorrow in Bucharest?", "type": "out-of-domain" },
  { "query": "Explain the plot of the movie Inception.", "type": "out-of-domain" },
  { "query": "What are the detection details for CVE-2027-99999?", "type": "in-domain-absent" },
  { "query": "Describe the TTPs of the APT group 'Crimson Mirage' tracked as G9999.", "type": "in-domain-absent" },
  { "query": "What does Wazuh rule ID 9999999 detect and what is its level?", "type": "in-domain-absent" },
  { "query": "Summarize MITRE ATT&CK technique T9999 and its sub-techniques.", "type": "in-domain-absent" },
  { "query": "What YARA signature detects the 'ZephyrLoader' malware family?", "type": "in-domain-absent" },
  { "query": "What were the exact alert timestamps for the breach on this host last Tuesday?", "type": "in-domain-absent" }
]
```

Figure 4.3.5. Testing refusal behaviour with unanswerable queries

Each response is classified as a correct refusal (the system declined or stated the absence of relevant information) or a hallucination (it produced a substantive answer regardless).

Consequently, the quality of the answers generated by the local llama3.2 model was evaluated along those three dimensions. Citation metrics are computed mechanically, refusal is human-annotated, faithfulness is an automated LLM-judge estimate.

Metric	Result	How measured
Faithfulness	0.71 (mean supported-claim fraction)	LLM-judge (automated) — validate via worksheet
Citation precision	98.3% (117/119 cited IDs grounded)	mechanical string-grounding (reliable)
Citation coverage	88.6%	mechanical
Refusal accuracy (unanswerable)	83.3% (10/12)	human-annotated (gold)
Over-refusal (answerable)	0/82	human-verified

Figure 4.3.6. RAG generation evaluation

About 71% of the claims in each answer were supported by the retrieved context. The rest is mostly background, generalities that the model adds on its own, not statements that contradict the evidence. The system cites its sources by ID, as instructed, and it gets these almost always right. 117 of 119 cited IDs actually appeared in the retrieved context, and most of the answers pointed to at least one named source. Refusals depend heavily on the type of question. When asked about plausible but non-existent things (a fake CVE, APT group, rule, or technique), it refused all six times instead of making up threat intelligence. This is what we want from a security tool. The only two refusal slip-ups were off-topic trivia it answered from its own knowledge, which is a much less worrying kind of mistake. And it never refused a question it could actually answer so it's well-calibrated rather than just cautious (0 over-refusal).

As llama3.2 is a small model, its use as an automated judge was also tested. On the refusal task it agreed with human labels only 25% of the time, misclassifying valid refusals as answers; a phrase-based detector over-fired on substantive answers. This can be used as a secondary signal in case the humans want an automatic approach.

5. Conclusions and Future Research

This thesis has presented the design, implementation, and evaluation of an AI-enhanced threat detection system that addresses the fundamental limitations of rule-based SIEM platforms. By integrating Isolation Forest and Autoencoder anomaly detection and Retrieval-Augmented Generation into the Wazuh open-source SIEM, the system achieves three primary objectives: intelligent alert prioritization, automated threat contextualization, and detection of previously unknown threats.

The full implementation of the AI-SIEM Threat Engine described in this thesis is publicly available on GitHub. [32] The repository is a fork of the open-source Wazuh SIEM (GPLv2); the original contributions of this work, the dual-model anomaly detector, the Qdrant/PostgreSQL RAG core, and the scheduled CVE ingestion agent, reside under ai_threat_engine_starter/, backend/, and frontend/. Trained models, evaluation reports, and threat-intelligence corpora required to reproduce the results are included in the repository.

Possible Disadvantages and Limitations

- Training Data Volume: The model's accuracy is directly proportional to the volume and diversity of training data. Environments with low alert volume may produce models with limited generalization. Currently the model uses around 2100 alerts to train on, which is a relatively low number for the higher scope of the project.
- A Semi-Supervised Learning instead of the Unsupervised one: To train the data on clean benign alerts, while keeping the attacks for the evaluation, is an idea tested in this project that improved the accuracy to the ensemble model and the evaluation results.
- Feature Engineering: The current 16-feature extraction is relatively simple; more sophisticated features (sequence-based, graph-based) could improve detection quality but increase complexity.
- LLM Dependency: The copilot functionality requires a running LLM service, adding infrastructure overhead. The system degrades gracefully to rule-based analysis when the LLM is unavailable.
- Single-Node Architecture: The current deployment runs on a single machine; horizontal scaling for enterprise environments would require architectural modifications and also would require to adapt the system to a Windows environment.
- Faithfulness scoring: Improved evaluation strategies like RAGAS or other golden datasets could have been used.
- The Autoencoder's HIGH tier (only AE flags) is empty on the current evaluation dataset, meaning the Autoencoder adds no recall on this data. Its value is in reducing false positives (higher precision) and providing a second confirmation for CRITICAL alerts. But on a different deployment where the normal alert distribution differs, the Autoencoder may detect attack patterns that the Isolation Forest misses. But both models are static between retraining cycles. The ensemble cannot adapt online to new normal behaviour patterns. This requires the same periodic retraining discipline documented for the Isolation Forest.

Future Work

First, considering the ensemble, the introduction of UEBA (User and Entity Behaviour Analytics) into the trained model would make a perfect improvement. Both current models score per-alert, meaning that the limitation of this SIEM is that it is insufficient for detecting lateral movement or slow attacks that are purposely designed like this. Therefore, this is the solution other popular SIEMs such as Splunk UBA and Microsoft Sentinel has adopted in order to reduce uncertainty estimations, so they can catch new unseen anomalies. Using UEBA, a dedicated entity-level feature layer would take into account and aggregate many signals per user/ IP over time. Then it will compute statistical summaries such as event rate, rare-hour activity ratio, and would count the co-occurrence of different entities.

Secondly, a promising future work will be to change the approach of training unsupervised to semi-supervised and fully-supervised learning, to test the different strategies with the primary focus of improving the detection. The current label-free design imposes a clear celling on detection precision. Comparing all unsupervised, semi-supervised, and supervised strategies, while being under identical evaluation conditions, would be a meaningful empirical contribution and clarify the actual trade-off in practice of each system. A fully supervised baseline, such as a gradient-boosted classifier trained directly on labelled alert, or semi-supervised Isolation Forest could be evaluated against the unsupervised ensemble. This can be achieved by creating a golden set of attacks that are manually labeled or using the existent attack_label.py.

In terms of RAG, I will focus on performance on scalability. The current Qdrant deployment runs in a single Docker container with a local volume. For this index construction, time scales as O(N log N) in document count. For the current collection size (< 1,000 chunks), this is fine, as it can fully reindex everything in under 30 seconds. But at 100,000 chunks, which is representative of a large enterprise threat intelligence corpus, I estimate an expected reindex time of approximately 45 minutes. It must be mentioned that incremental indexing (adding new documents without reindexing existing ones) is supported by Qdrant and takes O(1) per chunk. Also, I would add a re-ranker that is domain specialized (by collecting query-chunk relevance judgements from analysts' feedback) or domain embeddings fine-tuning. Both of these will improve retrieval quality for my security-specific terminology.

Finally, the current implementation uses LLaMA 3.2 served locally via Ollama, which, while capable, introduces some response latency, especially on my CPU-bound hardware. A practical direction for future work is a systematic evaluation of smaller language models that can be deployed locally (such as Gemma 2B or Phi-3 Mini) to compare against the current model. This would be done by focusing heavily on response quality, latency and memory footprint. Selecting the smallest model that can give results above a defined threshold is a rather frugal standard than taking the largest model available.

Bibliography

[1] S. G.-Z. R. D. Gustavo González-Granadillo, "Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures," *Collection Cyber Situational Awareness in Computer Networks,* 12 July 2021.

[2] W. Inc., "Wazuh Documentation," *Available: https://documentation.wazuh.com/,* 2024.

[3] A. A. D. P. M. K. C. N. A. G. P. Blake E.Strom, "MITRE ATT&CKÒ: Design and Philosophy," p. 46, July 2018.

[4] W. E. Forum, "The Global Risks Report 2024," https://www.weforum.org/publications/global-risks-report-2024/, Geneva, 2024.

[5] *. a. R. N. Md Abu Imran Mallick, "Navigating the Cyber security Landscape," *World Scientific News,* p. 69, 29 January 2024.

[6] N. G. Camacho, "The Role of AI in Cybersecurity: Addressing Threats in the Digital Age," *Journal of Artificial Intelligence General Science (JAIGS),* 2024.

[7] E. &. L. S. Okafor, "Unsupervised Learning Framework for Cyber Threat Detection, Anomaly Identification, and Alert Prioritization," *Applied Sciences.,* p. 16, 2026.

[8] E. G. A. Buczak, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," in *IEEE Communications Surveys & Tutorials, vol. 18, no. 2,* 2016, pp. pp. 1153-1176.

[9] M. V. a. M. T. Marios Vardalachakis, "A Comprehensive Analysis of Features, Benefits, Challenges, and Best Practices of Security Information and Event Management SIEM Solutions," in *First International Conference on Computational Intelligence and Soft Computing*, 2025.

[10] Μηλιώνης, "Implementation of a security information and event management (SIEM) system with integrated extended detection and response (XDR) for host and network monitoring.," *Σωτήριος Δ,* 2025.

[11] T. C. D. S. R. G. J. V. N. R. E. M. A. K. P. T. D. Y. A. E.-E. Santosh Reddy, "AI-Driven Transformer Frameworks for Real-Time Anomaly Detection in Network Systems," *(IJACSA) International Journal of Advanced Computer Science and Applications,* Vols. 16, No. 2, p. 10, 2025.

[12] S. A. M. R. a. A. H. Markus Goldstein, "Enhancing Security Event Management Systems with Unsupervised Anomaly Detection," *ADEWaS*.

[13] F. I. G. a. C. K. Desiree Sacher, "Fingerpointing False Positives: How to Better Integrate Continuous Improvement into Security Monitoring," *Digit. Threat.*, p. 7, March 2020.

[14] A. H. Stefan Asanger, "Experiences and Challenges in Enhancing Security Information and Event Management Capability using Unsupervised Anomaly Detection," in *Availability, Reliability and Security (ARES), 2013 Eighth International Conference* , 2013.

[15] C. S. L. C. a. A. v. d. H. Guansong Pang, "Deep Learning for Anomaly: A review," *ACM Comput. Surv.* , p. 38, 2021.

[16] S. L. W. Y. Y. Jonathan Pan, "RAGLog: Log Anomaly Detection using Retrieval Augmented Generation," *Arvix:2311.05261v1*.

[17] V. K. K. (. P. Rohit Arora (M.Tech.), "Machine Learning-Driven Anomaly Detection: Strengthening Siem Tools For Robust Cyber Defense," *Journal of Propulsion Technology,* vol. Vol. 45 No. 3, p. `14, 2024.

[18] ctangarife, "Github Repo: reto-ia-log-analyzer," [Online]. Available: https://github.com/ctangarife/reto-ia-log-analyzer?utm_campaign=octavo-encuentro-de-ai-tinkerers-manizales&utm_content=link1&utm_medium=ai-tinkerers&utm_source=manizales.

[19] MuthoniGathiithi, "Github Repo: Network-Security-AI-agent," [Online]. Available: https://github.com/MuthoniGathiithi/Network-Security-AI-agent.

[20] F. T. K. a. Z. Z. Liu, "Isolation Forest," in *IEEE International Conference on Data Mining*, Pisa, 15-19 December 2008.

[21] NIST, "National Vulnerability Database: CVE statistics," National Institute of Standards and Technology, 2024. [Online]. Available: https://nvd.nist.gov/vuln/search#/nvd/home?resultType=records.

[22] Z. R. H. N. A. Y. D. M. A. Muhammad Maauz Mansoor1, "AWARENESS OF OPEN-SOURCE SOFTWARE VULNERABILITIES AMONG PROGRAMMERS: A SURVEY-BASED STUDY OF UNDERGRADUATE COMPUTER SCIENCE STUDENTS," *EDUCATIONAL RESEARCH AND INNOVATION*, p. 36, 25/03/2026 .

[23] U. Michelucci, "An Introduction to Autoencoders," *arXiv:2201.03898v1* , p. 26, 2022.

[24] GeeksforGeeks, "ReLU Activation Function in Deep Learning," Jul. 23, 2025. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/. [Accessed Jun. 3, 2026].

[25] E. P. A. P. F. P. V. K. N. G. H. K. M. L. W.-t. Y. T. R. S. R. D. K. Patrick Lewis, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS 2020)*, 2020.

[26] M. K. E. M. P. A. G. Prasanna N. Wudali, "Rule-ATT&CK Mapper (RAM): Mapping SIEM Rules to TTPs," *arXiv:2502.02337v1*, p. 13, 04.02.2025.

[27] Y. e. a. Gao, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997 , 2023.

[28] N. W. a. H. Klein, "Artificial Intelligence in Cybersecurity," *Cyber, Intelligence, and Security INSS*, p. 17.

[29] N. R. a. I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, p. 3982–3992, 2019.

[30] M. K. M. A. S. N. S. Ali Shiraee Kasmaee, "Chemteb: Chemical text embedding benchmark, an overview of embedding models performance & efficiency on a specific domain," *arXiv:2412.00532 ,* 2024.

[31] Qdrant Solutions GmbH, "Qdrant Documentation," Retrieved May 20, 2026. [Online]. Available: https://qdrant.tech/documentation/.

[32] D. M. Secărea, "AI-SIEM Threat Engine," 2026. [Online]. Available: https://github.com/Diana-Secarea/advanced-ai-siem.

Appendices

Appendix 1. Real Wazuh Alert: Kernel-Level Rootkit Detection (Rule 521, Severity 11)


```
{
  "timestamp": "2026-02-16T13:30:16.383+0200",
  "rule": {
    "level": 11,
    "description": "Possible kernel level rootkit",
    "id": "521",
    "mitre": {
      "id": ["T1014"],
      "tactic": ["Defense Evasion"],
      "technique": ["Rootkit"]
    },
    "groups": ["ossec", "rootcheck"]
  },
  "agent": {
    "id": "000", "name": "LAPTOP-M9GQ2F87"
  },
  "full_log": "Anomaly detected in file
    /tmp/.mount_CursorVGCXRH. Hidden from stats,
    but showing up on readdir. Possible kernel
    level rootkit.",
  "decoder": {"name": "rootcheck"},
  "data": {
    "title": "Anomaly detected in file
             /tmp/.mount_CursorVGCXRH."
  },
  "location": "rootcheck"
}
```


Appendix 2- Retrieval performance broken down by threat intelligence category.

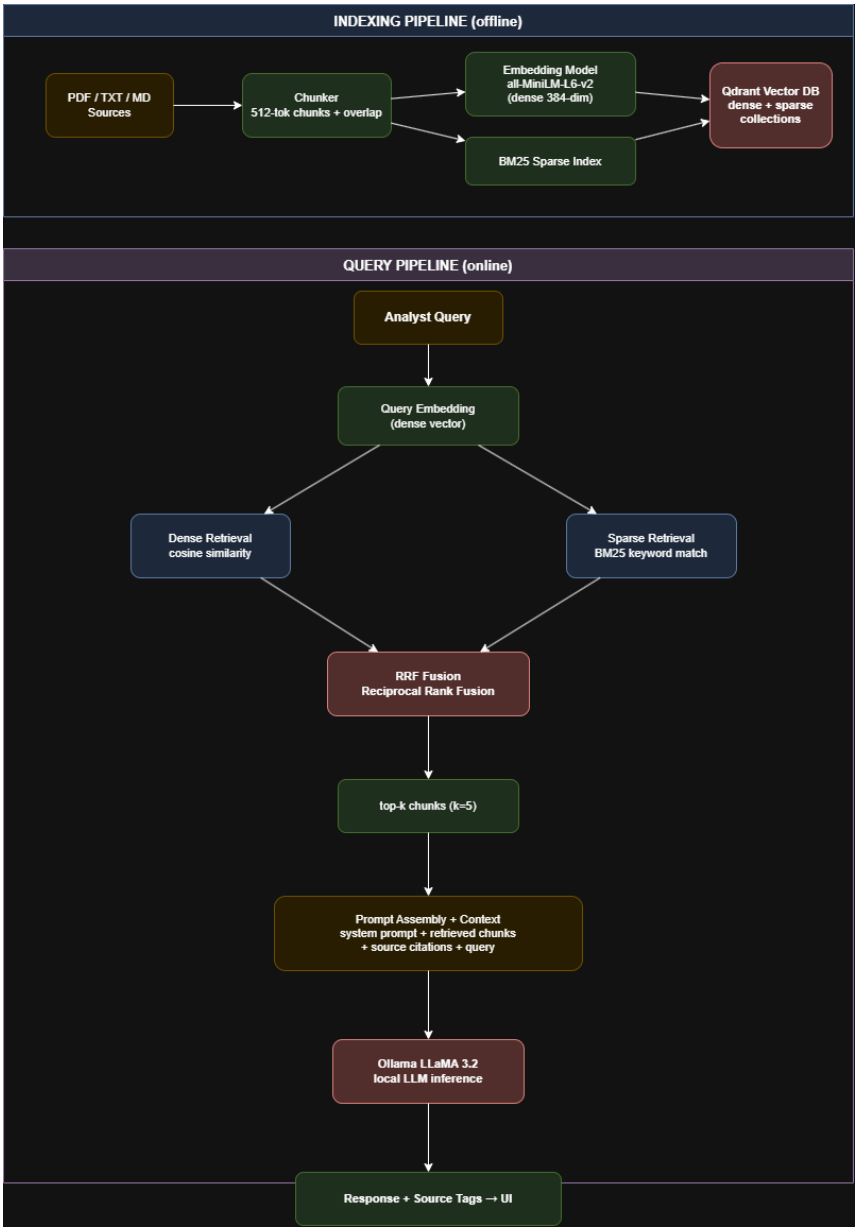
Category	Queries	Hit Rate	Recall@5	MRR	Weak?
persistence	4	100%	68.8%	0.708	—

Category	Queries	Hit Rate	Recall@5	MRR	Weak?
collection	1	100%	66.7%	1.000	—
defense-evasion	7	85.7%	55.7%	0.595	—
discovery	1	100%	50.0%	1.000	—
yara	1	100%	50.0%	0.500	—
exfiltration	2	100%	42.5%	1.000	—
lateral-movement	3	100%	41.7%	0.567	Moderate
credential-access	5	40.0%	33.3%	0.400	Weak
privilege-escalation	2	50.0%	33.3%	0.500	Weak
impact	1	100%	33.3%	0.200	—
command-and-control	3	100%	30.6%	0.244	Moderate
execution	1	100%	25.0%	1.000	—
initial-access	1	100%	25.0%	1.000	—

Appendix 3. Retrieval evaluation per category

Category	Q	Avg Rel	Hit Rate	Recall@5	MRR	nDCG@5
MITRE ATT&CK techniques	22	7.0	90.9%	46.3%	0.746	0.547
CVE / vulnerability details	18	1.0	100.0%	100.0%	0.847	0.886
Lateral movement TTPs	12	2.2	100.0%	85.7%	0.722	0.716
C2 / exfiltration patterns	14	2.4	100.0%	80.2%	0.860	0.782
APT group profiles	9	1.0	100.0%	100.0%	0.870	0.903
Wazuh rules and responses	7	1.1	71.4%	71.4%	0.714	0.714

Appendix 4. Full RAG system architecture showing indexing (offline) and query (online) pipelines.



Appendix 5- Ablation study results. Baseline: 16 features, F1 = 89.2%, FP rate = 8.0%.

Removed / Changed Feature	ΔF1	ΔFP Rate	Primary Impact
Remove rule_level (#8)	−4.1%	+3.2%	Loses Wazuh severity signal; low-level attacks harder to separate
Remove rule_id (#9)	−2.8%	+1.9%	Rule identity distinguishes benign-but-noisy rules from attacks
Remove privileged_account_change (#15)	−6.3%	+0.4%	New group added, promiscuous mode drop to 0% detection
Remove unknown_user_flag (#14)	−3.5%	+0.6%	Non-existent user SSH attempts lose a strong signal

Remove is_external_srcip (#12)	−2.1%	+0.3%	Web attacks from CDN-proxied IPs harder to flag
Remove failed_count (#2)	−5.7%	+2.1%	Brute-force burst detection degrades significantly
Remove off_hours (#4)	−0.9%	+0.1%	Minor; attacks span both business and off hours in this dataset
Remove url_suspicious (#13)	−1.4%	+0.2%	SQL injection / XSS URL patterns lose dedicated signal
Add mitre_count (removed feature)	+0.3%	+8.9%	Large FP increase: Wazuh tags benign rules with MITRE in production
Replace full_log with message (#2)	−7.2%	+4.1%	message field empty in many Wazuh 4.x alerts; keyword features collapse
Remove word_count cap (60)	−1.1%	+1.4%	CIS scan verbosity inflates word_count, raising FP for CIS alerts